PROVISIONING AND INSTALLATION GUIDE OF THE K2VIEW AZURE MARKETPLACE APPLICATION

K2view Data Product Platform Services

Version: 1.3

Date: 6 November 2024

Overview

This guide provides a comprehensive walkthrough for provisioning and installing the K2view Data Product Platform in a self-hosted environment on Microsoft Azure. Targeted for IT and DevOps audiences, it details each phase of the installation process - from the steps and prerequisites to install the K2view Azure Application resources to deploying and configuring the K2view's cloud services within your self-hosted environment. The guide addresses setting up Kubernetes clusters, managing resource groups, and ensuring connectivity to customer-managed services.

This documentation outlines necessary prerequisites, roles, and post-installation validation steps. Please use the prerequisites and installation checklist to prepare and plan your installation.

Should a problem arise during the installation, please consult with your K2view representative to obtain support.

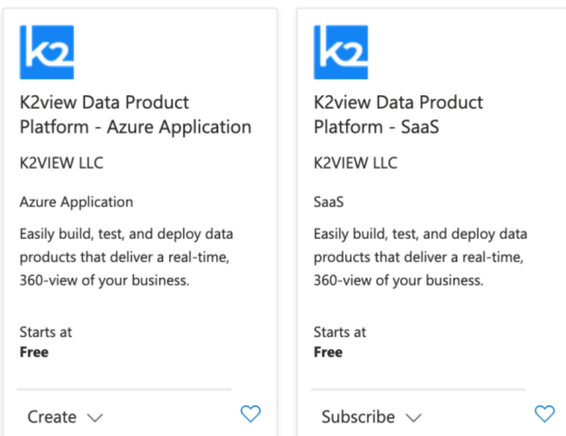
Table of Contents

1. The End State

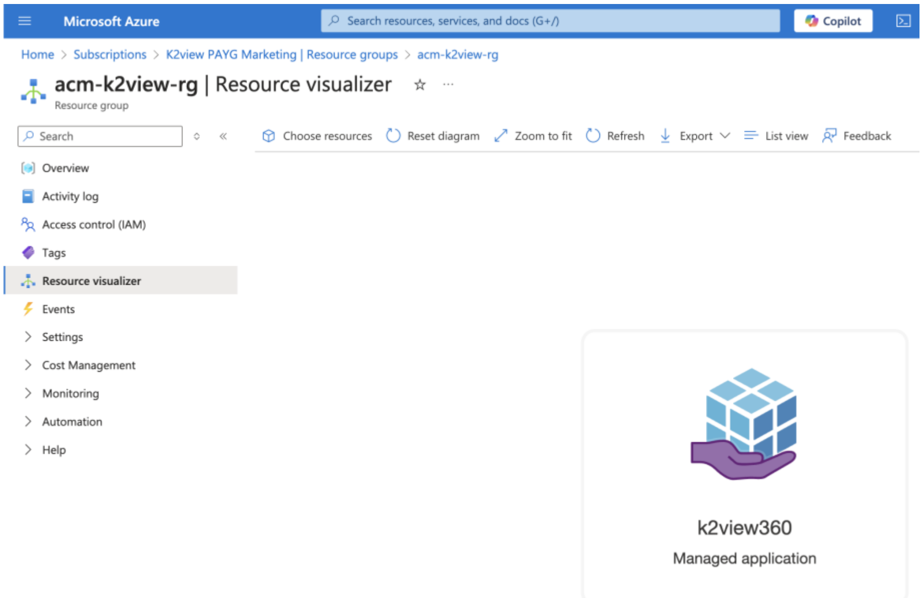
Upon completing the instructions in this guide, the K2view Data Product Platform will be fully deployed and operational within your Azure environment. This includes a fully configured Azure Kubernetes Service (AKS) cluster, secured network components, and connected resource groups, allowing you to start building and managing data products. The installed environment will be ready for integration with your existing data sources and K2view's cloud services, enabling the creation of projects and spaces for data management. Additionally, your Azure Container Registry will be populated with the necessary K2view Docker images, setting the foundation for scalable and secure operations

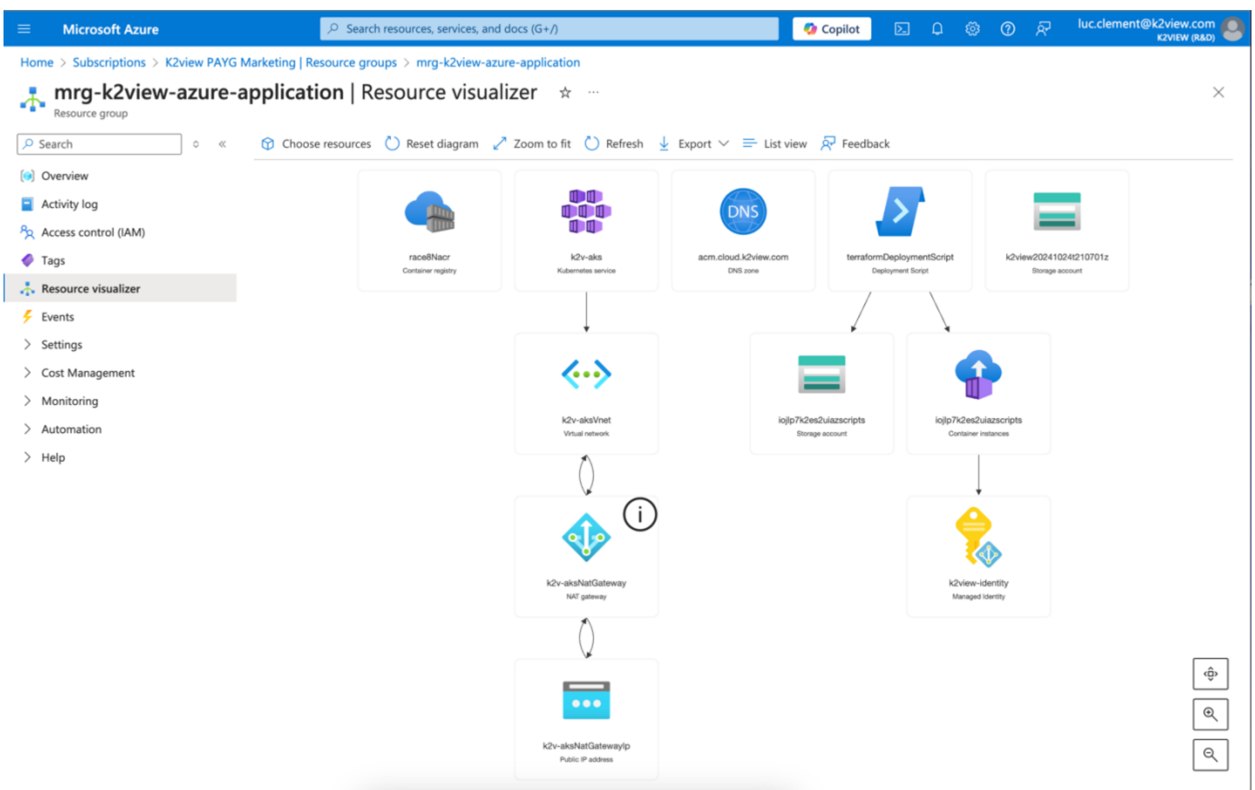
The Planning section will guide you with the information you need to gather to complete the installation.

The process starts by selecting the **K2view Data Product Platform—Azure Application** listing on the Microsoft Azure Marketplace. This listing will be used to install the K2view Data Product Platform within your Azure environment.

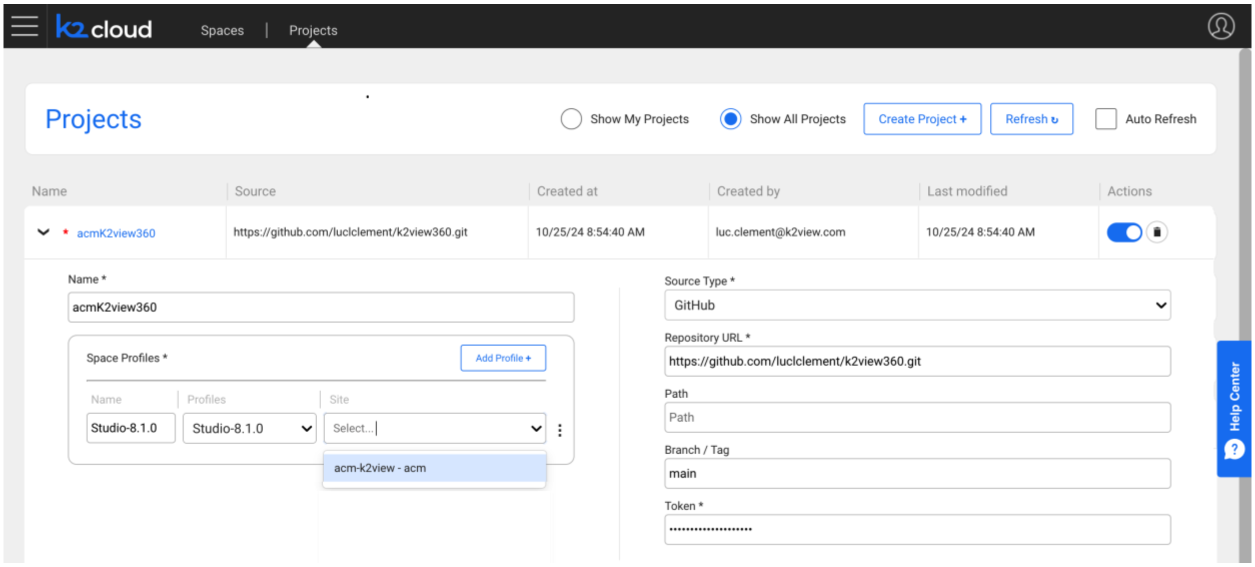


Upon completing the installation two resource groups will have been created: one for the K2view Managed Application and the other for the Kubernetes and infrastructure services on which the K2view Data Product Platform services will run.

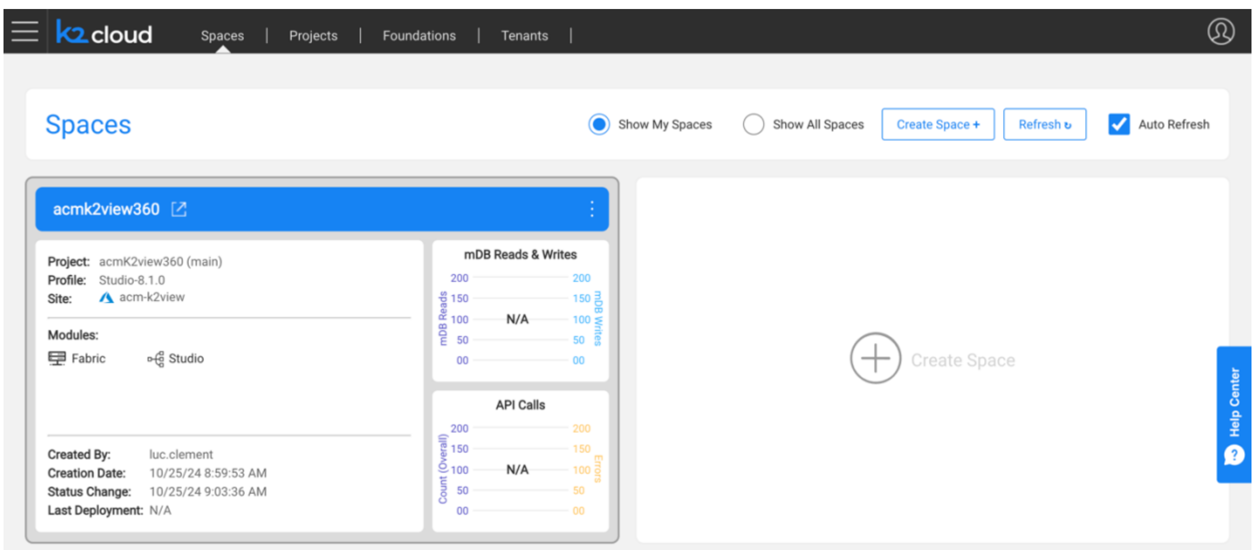




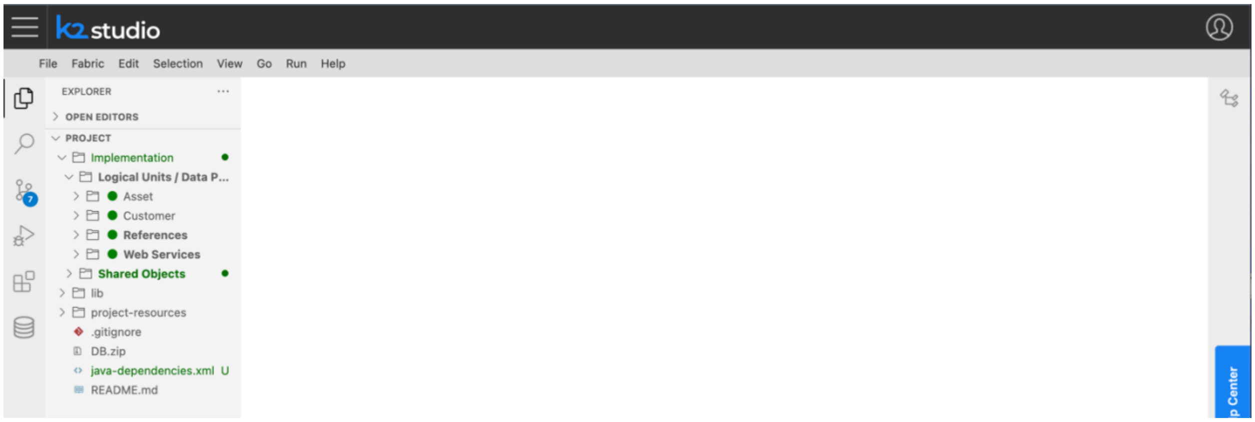
Upon completion of the installation, you will create your K2view project.



And then, you will create your first K2view Space.



And finally, you will be ready to start your first project, and start learning using our Knowledge Base at https://support.k2view.com/knowledge-base.html, where you can find documentation and demo projects, and sign up for the K2view Academy Learning Center at https://academy.k2view.com.



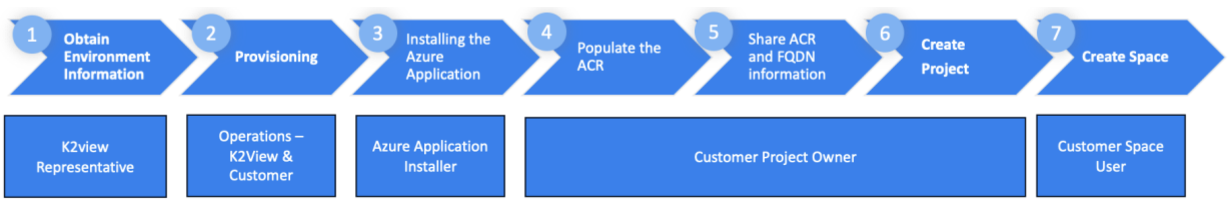
Next, let's plan this installation.

2. Planning

Planning and capturing the necessary information prior to the installation will help ensure a smooth installation experience. The customer and K2view will gather and share information used to provision the environment.

This section summarizes the steps, constraints, and prerequisites. At the end of the section, you will find an Installation Checklist and an Installation Worksheet that will help you ensure you have everything on hand to complete the installation.

2.1. Steps



Step 1 – Obtaining Environment Information

A K2view representative working with the customer will discuss the prerequisites to complete the installation and review the Installation Checklist.

Step 2 - Provisioning

During the provisioning step:

- K2view will create Nexus and user accounts, create a tenant, and create associated configurations. Some of the information that K2view needs to complete the configuration will need to be provided by the customer in Step 5.
- The customer will gather information based found in the Installation Checklist such as Git details, a TLS certificate (PEM encoded) and its private key (PEM encoded), and names of entities. The certificate and private key files need to be base64 encoded. The customer will also need to validate prerequisites for the user installing the K2view Azure Application within the Azure subscription.

Step 3 – Install the K2view Azure Application

A person with the necessary roles – the Subscription Owner role - within the Azure subscription will run the K2view Azure Application installation.

Step 4 – Populate the Azure Container Registry with K2view Docker images

The K2view project owner will load the selected Azure Container Registry with K2view Docker images that have been identified as required for the customer during Step 1.

Step 5 – Share ACR and Domain Name information with K2view

To complete the configuration for the tenant's creation of Projects and Spaces, the Customer Project Owner will need to share the following information used.

- The fully qualified domain name (FQDN) used for your Domain Ingress
- The list of Azure Container Registry (ACR) locations of your Docker images
- The AKS Cluster name

This information may have been identified during pre-planning, but the specific information should be confirmed to avoid misconfiguration.

Step 6 - Create the K2view Project

Step 7 - Create a K2view Space

2.2. Constraints

There are three constraints you need to be aware of to install the K2view Azure Application within your Azure subscription.

- **Azure Subscription Type**. You will be installing the K2view Azure Application in an Azure Subscription that needs to be a Pay-As-You-Go (PAYG) or Enterprise Agreement subscription.
- **Subscription Owner Role.** The installer of the K2view Azure Application needs specific Subscription level roles. The user needs to be a **Subscription Owner**.
- **Azure Resource Registration**. Azure resources must be registered before the installation by a **Subscription Owner**. The list of resources and commands to use are provided in the Azure Resource Registration section.
- **Region and availability zones**. Please note that the installation requires you select an Azure region that offers more than one availability zones. Not all Azure Regions support multiple availability zone.

2.3. Prerequisites

Certain prerequisites must be met, and planning steps must be taken to install the K2view Azure Application, set up the Azure Container Registry, and create a K2view Project and a Space.

These prerequisites are summarized here.

- You need to verify that your Azure Subscription is a PAYG or an Enterprise Agreement subscription.
- You need the person performing the K2view Azure Application installation to have a **Subscription-level Owner** role.
- Azure resources must be registered before the installation by a **Subscription Owner** (see: Azure Resource Registration).
- A Git code repository is required for your Fabric Project. Three pieces of information will be required to create your K2view project including:
    - Repository URL
    - Git token
    - Git Branch / Tag

  The creation of a Git repository, if one is not already available for your use, is described in the GitHub section.

- You will need access to K2view's Nexus Repository and Docker installed on a machine used to populate the Azure Container Registry (ACR) that will be created within your Azure Resource Group.
    - The installation of the K2view Azure Application will create an ACR; if you plan to use a pre-existing ACR you may do so. Whether you use the ACR or an OCI-compatible container registry the location where you will be populating K2view Docker images needs to be provided to K2view as described in Step 5 above and in the Sharing information with K2view section.
    - You need to populate your selected ACR by pulling and pushing docker images.
        1. Docker needs to be installed on a computer to pull and push docker images from the K2view Nexus repository to the ACR created
        2. You will need a K2view Nexus account you can obtain from your K2view representative

  The Populating Your Azure Container Registry section describes the steps.

- Pre-planning names will help a smooth installation. Use the Checklist and capture this information in a document. This includes:
    - The Azure region to be used. Please note that the installation requires an Azure Region that provides two or more availability zones. Not all Azure Regions have multiple availability zones.
    - The names of the two resource groups where K2view Fabric services will be installed.
        1. **Azure Application Resource Group**. You can choose an existing resource group or create a new one. You should select a representative name. In the example

demonstrated in this guide we use "acm-k2view-rg". Select something that has significance to your project.

2. **Azure Managed Application Resource Group**. K2view resources will be created in a separate resource group, the Azure Managed Application resource group. This resource group contains K2view infrastructure components created during the installation. A name will be suggested for you. You can use that name or one that has significance to you. In our example, we used **mrg-k2view-azure-application.**

- **Cluster Name**: This is an Azure Kubernetes Service (AKS) cluster name of the Kubernetes cluster to be created. The default, if used, is k2v-aks.
- **Environment Tag**: This tag describes the project's environment. Typical values could be Dev, UAT, and Prod. Azure cost management modules will use this tag to help you categorize costs.
- **Project Tag**: This tag describes the functional purpose of the project. You might consider tag names such as K2view TDM, K2view Masking, and K2view 360. Azure cost management modules will use this tag to help you categorize costs.
- **Domain Ingress**: This is the FQDN of the Nginx Ingress Controller for which a PEM wildcard certificate needs to be created. In our example, we used acm.cloud.k2view.com, a domain owned by K2view. You will need to select your own domain and one for which you can create TLS certificates (PEM encoded).

- TLS keys and certificates

  Your network team must create a wildcard TLS certificate for the domain you selected as your Domain Ingress. The full certificate authority chain needs to be present. During the installation, you will need to paste the base64 values of your PEM TLS certificate and its associated private key.

  - **TLS Private Key**: the base64-encoded PEM private key associated with the TLS certificate
  - **TLS Certificate**: a base64-encoded PEM wildcard certificate. For example, for the domain ingress of acm.cloud.k2view.com you need to create a wildcard certificate for *.acm.cloud.k2view.com for hosts of this domain.

    These should look like the following, respectively (shortened for brevity and non-disclosure of sensitive):

    PEM TLS Private Key:

```
-----BEGIN PRIVATE KEY-----
MIIJQwIBADANBgkqhkiG9w0BAQEFAASCCS0wggkpAgEAAoICAQC1CFTpFyQaTglb
…
fHZLomEYrUhsv9X+RUz6f3WnzzbfXCA=
-----END PRIVATE KEY-----
```

    PEM TLS Certificate:

```
-----BEGIN CERTIFICATE-----
MIIFUjCCAzoCCQDjx3pKVc6ytTANBgkqhkiG9w0BAQsFADBrMQswCQYDVQQGEwJV
…
YfROGHT8lE1aDUh17MHvRbnHI4qekw==
-----END CERTIFICATE-----
```

The PEM certificate and key must be further encoded using base64 as input to the installation screen. To encode a PEM certificate (e.g., cert.pem) to base64 on both Linux and Windows, you can use the following commands.

Linux: You can use the base64 command to read the cert.pem file, encodes it in base64, and saves the output to cert_base64.txt.

```
base64 cert.pem > cert_base64.txt
```

Windows: On Windows, you can use the certutil command to encode the cert.pem file in base64 and saves the result to cert_base64.txt.

```
certutil -encode cert.pem cert_base64.txt
```

The base64-encoded versions of the private key and certificate should look like the following:

base64-encoded TLS Private Key:

```
LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0tCk1JSUpRd0lCQURBTkJna3Foa2lHOXcwQkFRRUZBQVNDQ1
Mwd2dna3BBZ0VBQW9JQ0FRRE9yakUrN0NQVkU2QVgKV0lFRFVvdnVLc0lXOEJ4MTR
L…
QTlZV1pKKzlqRkhFUTdBZXB3S1RtMEVraz0KLS0tLS1FTkQgUFJJVkFURSBLRVktLS0tLQo=
```

base64-encoded TLS Certificate:

```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUdGRENDQlB5Z0F3SUJBZ0lTQS9YVFJKaUpXOXMzbl
dZZlN5a0RpZlh5TUEwR0NTcUdTSIzRFFFQkN3VUEKTURNeEN6QUpCZ05WQkFZVEFsVlRNUll3RkFZRFZR
UUtFdzFNWlhRbm
…
1NDOKLS0tLS1FTkQgQ0VSVElGSUNBVEUtLS0tLQ==
```

Prerequisite: You should prepare these base64 encoded files before the start of the installation.

- You need to have been provisioned by K2view. Provisioning includes:
  - Creation of a tenant and configurations for you
  - Creation of an initial K2view Admin user
  - Creation of a K2view Nexus repository account
  - Creation of a K2view Cloud Mailbox ID

  In consultation with K2view, Docker images and their associated versions will be recommended. You will need this information to use docker pull commands to obtain images such as fabric, fabric-studio, and the k2-cloud-deployer images. These commands are described in the Populating Your Azure Container Registry section.

  K2view may provide you with access to k2echange.k2view.com optionally. Please discuss your needs with your K2view representative.

- After installing your K2view Azure Application, you will need to pause to provide K2view with information that will allow you to finalize your environment.
  - After you complete the installation and before you create a K2view project and a space, you will need to provide K2view with the location images within your container registry.
  - The list of images is those you will push as described in the Commands to Push Containers to the Azure Container Registry
  - For example, if you pushed fabric-studio:8.1.0_199 to the race8nacr.azurecr.io container registry used in this guide's examples, you will need to provide K2view with this value: race8nacr.azurecr.io/k2view/fabric-studio:8.1.0_199
  - Provide the location for every image pushed to your container registry.

2.4. Installation Checklist

Here is a checklist to help you gather the values and content you should have before the installation:

1. Identify an individual who will install the K2view Azure Application. This individual needs to hold a Subscription Owner role.
2. Azure resources need to be registered before the installation. This individual needs the Subscription Owner role and can be the same person performing the installation.
3. Identify the Azure Region you will be installing the K2view Azure Application, and confirm that it supports multiple availability zones
4. During the installation, you will need to enter the following:
    a. An Azure Application Resource Group name
    b. An Azure Managed Application Resource Group name
    c. An AKS Cluster name
    d. An Environment Tag name
    e. A Project Tag name
    f. Your Domain Ingress FQDN
    g. Obtained a Wildcard TLS PEM certificate for the Domain Ingress FQDN
    h. Obtained the corresponding PEM private key
    i. Base64-encode both the certificate and private key files
    j. The K2view Cloud Mailbox ID provided to you by K2view
5. To populate your Azure Container Repository, you will need
    a. K2view Nexus Repository Account supplied to you by K2view
    b. Obtained the list of Docker images to pull from Nexus and push to in consultation with K2view
    c. Docker installed on the computer used to pull and push Docker images to your ACR
6. After the image population of your ACR is complete, and before the creation of your K2view Project and your first Space, you need to provide K2view with. Once confirmed you will be able to create your K2view Project:
    a. The FQDN of your Domain Ingress
    b. The list of ACR locations of your Docker images
    c. The AKS Cluster name
7. To create your K2view project, you will need:
    a. A Git Repository URL
    b. Git token
    c. Git Branch / Tag
    d. Your initial K2view Admin user

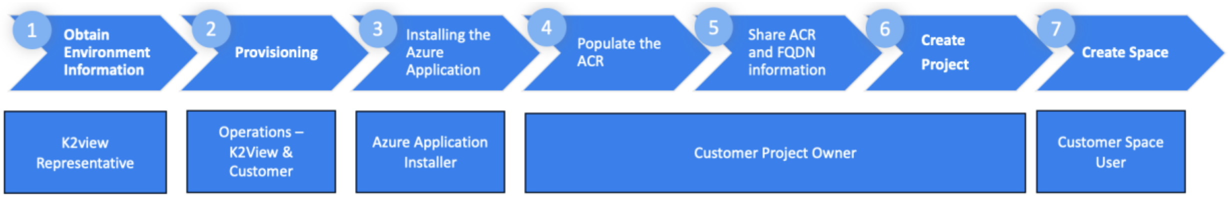
2.5. Obtaining and Capturing Required Information

The Installation Worksheet at the end of this document will help you to capture the information you need to collect to complete the installation. Print these pages to capture the information you will need to have on hand to perform the installation.

2.6. Post Installation

- Validation of the environment should be performed, including the DNS zone configuration. Please see the DNS zone section.
- Should SSO and IDP federation be desired, following the initial installation, additional information will be required and planned in collaboration with K2view.

3. Installing the K2view Azure Application



This section describes Step 3 – The Installation of the K2view Azure Application.

The prerequisites of this section are:

1. Identify an individual who will install the K2view Azure Application. This individual needs to hold a Subscription Owner role.
2. Azure resources need to be registered before the installation. This individual needs the Subscription Owner role and can be the same person performing the installation.
3. Identify the Azure Region where you will be installing the K2view Azure Application and confirm that it supports multiple availability zones.
4. During the installation, you will need to enter the following:
    a. An Azure Application Resource Group name
    b. An Azure Managed Application Resource Group name
    c. An AKS Cluster name
    d. An Environment Tag name
    e. A Project Tag name
    f. Your Domain Ingress FQDN
    g. Obtained a Wildcard TLS PEM certificate for the Domain Ingress FQDN
    h. Obtained the corresponding PEM private key
    i. Base64 encoded both versions of the certificate and private key files
    j. The K2view Cloud Mailbox ID provided to you by K2view

Before you begin the installation process

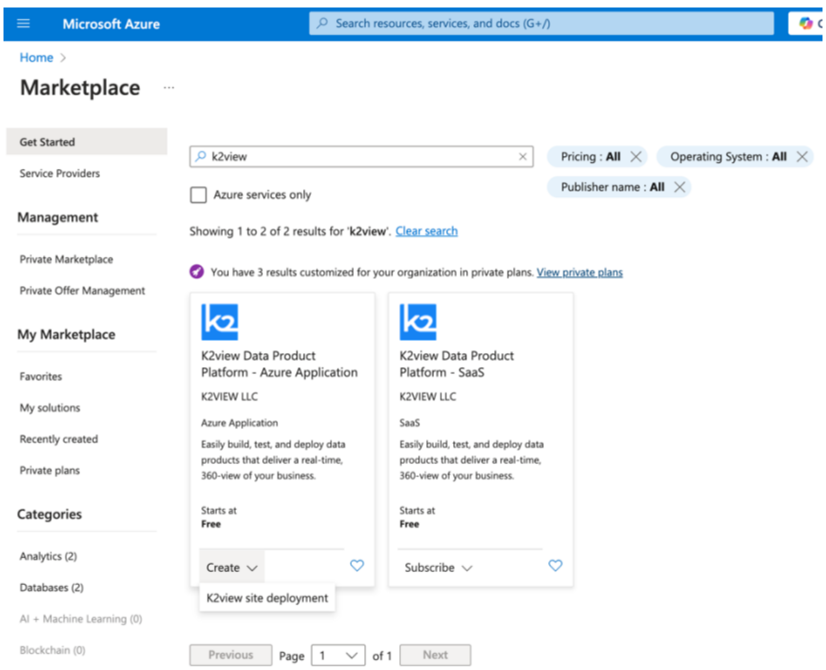
Azure resources need to be registered before the installation as described in the Azure Resource Registration section.

Proceeding to the installation

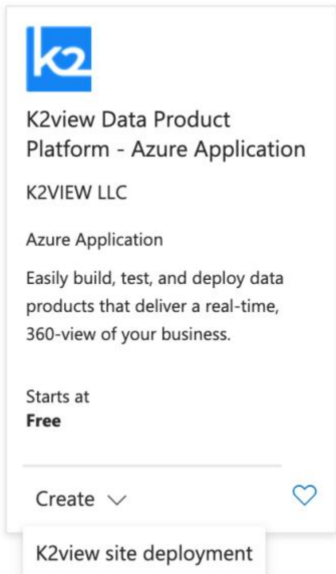
The Azure Application installer is available and found on the Azure Marketplace. Search for "K2view" in the marketplace as shown below. For the purpose of this installation you will use the "K2vew Data Product Platform – Azure Application.

This application will fully install Fabric services within a resource group of your Azure subscription. The installation process is fully automated and requires only minimal input provided in the Installation Checklist section. Please review the Planning in its entirety to understand the constraints, prerequisites, and the meaning of the terms used.

Search for K2view



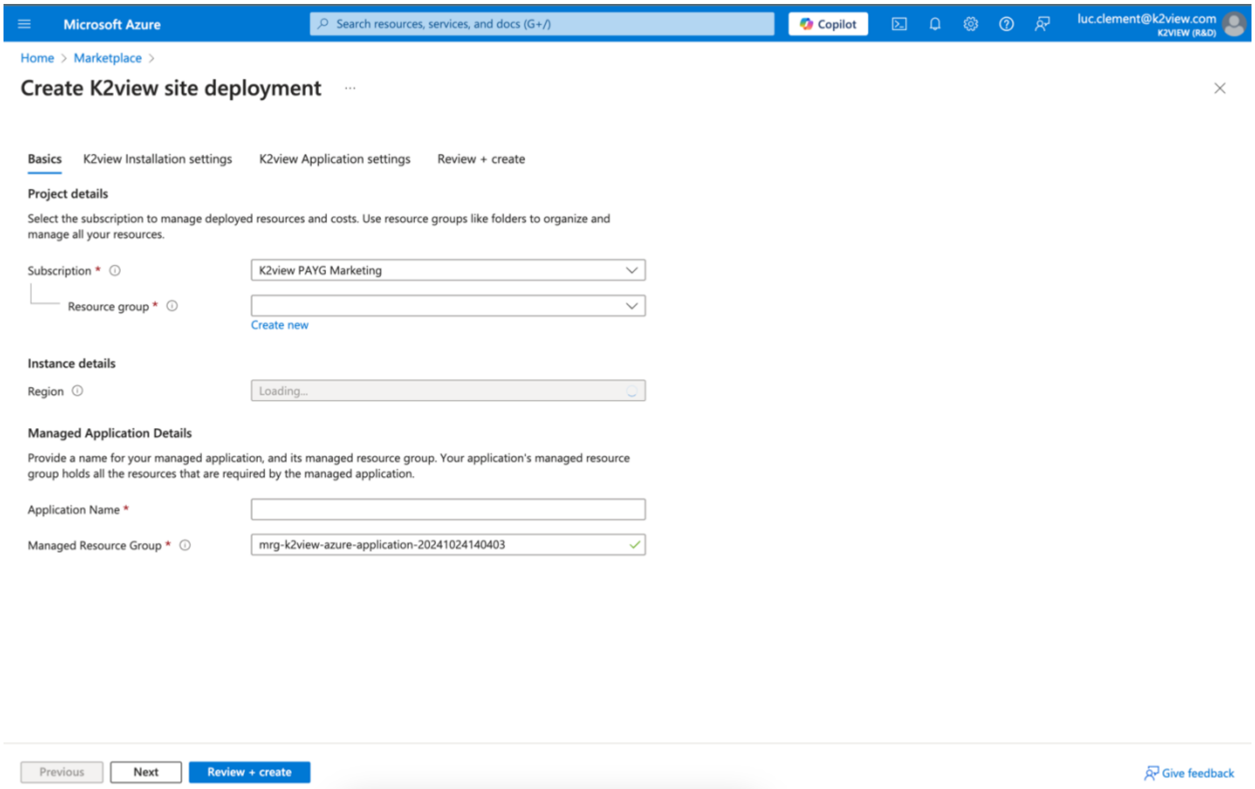
Select the Azure Application listing and press Create.

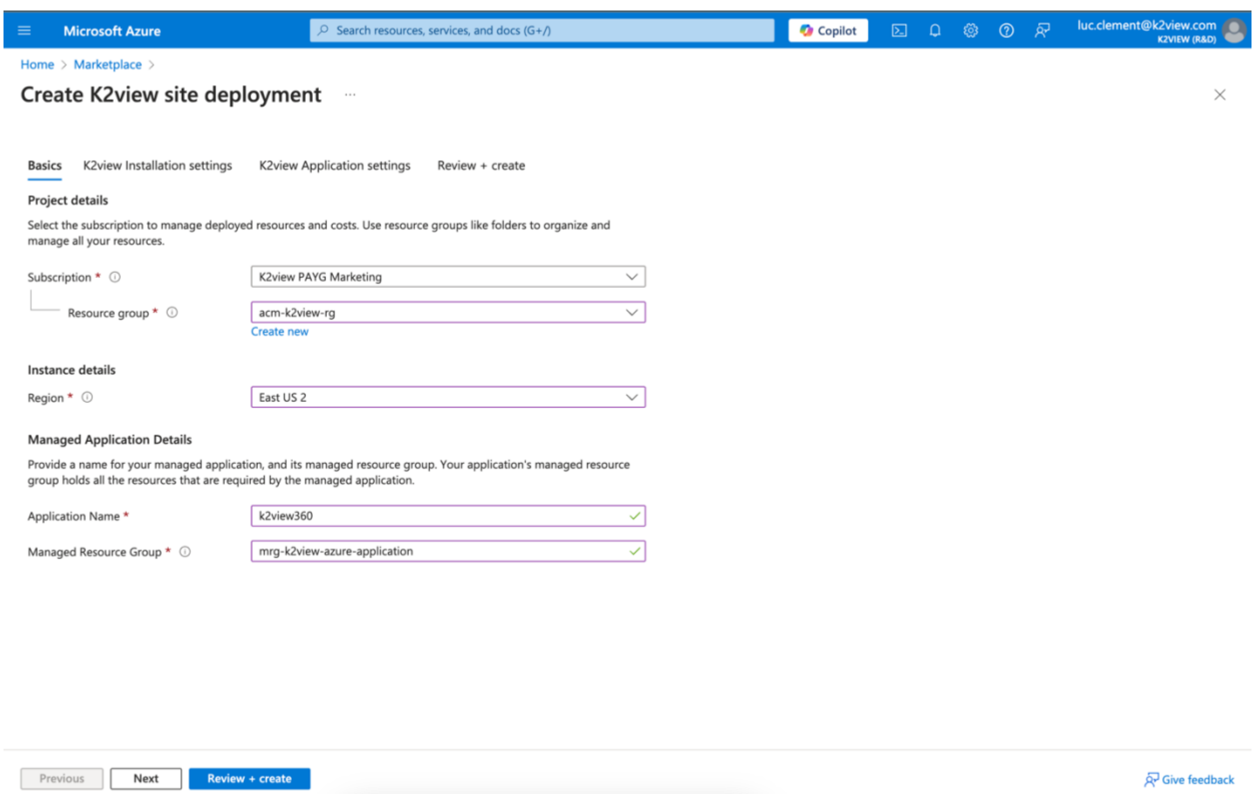


The installation process will guide you through several screens. You must select the appropriate Azure subscription, a resource group that can be a preexisting one or one you create, and the region where the Fabric services will be installed. Additionally, you must provide the name of the Azure-managed application and its associated resource group.

Please consult the Constraints and Prerequisites section to ensure you have been assigned the Subscription Owner role and identified an Azure region that supports multiple availability zones.

First Page of the Installation Wizard





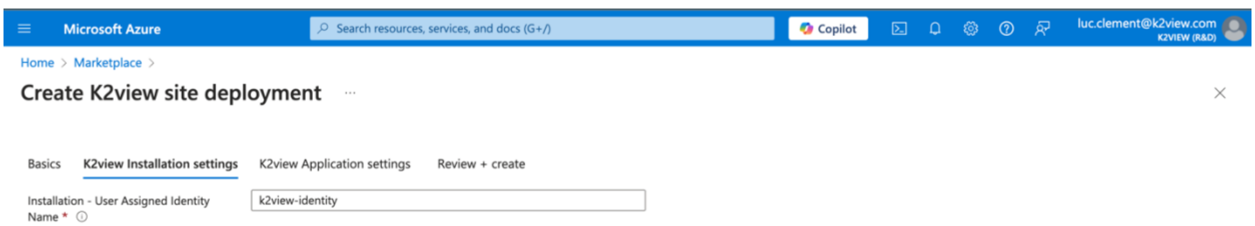
The installation uses two resource groups.

- **Azure Application Resource Group**. This is shown on the page below "Subscription". You can choose an existing resource group or create a new one. You should select a representative name. In the example demonstrated in this guide we use "acm-k2view-rg". Select something that has significance to your project.
- **Azure Managed Application Resource Group**. This is shown on the page as "Managed Resource Group". K2view resources will be created in a separate resource group, the Azure Managed Application resource group. This resource group contains K2view infrastructure components created during the installation. A name will be suggested for you. You can use that name or one that has significance to you. In our example, we used **mrg-k2view-azure-application.**

Press Next at the bottom of the page.

Second Page of the Installation Wizard

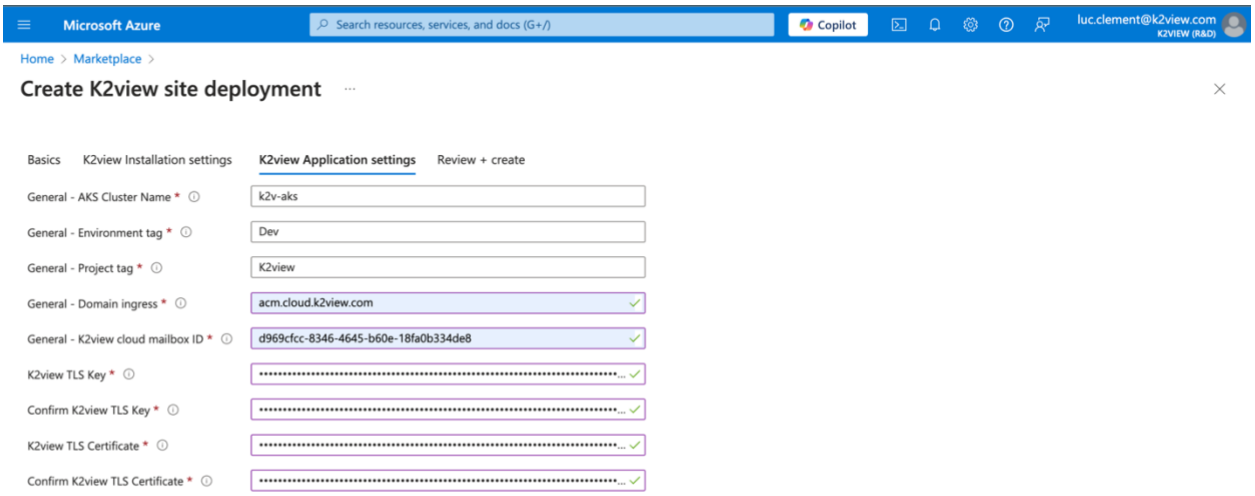
On this screen you set the identity used for the installation. Unless you have specific needs to do otherwise, we recommend that you use the default of "k2view-identity"



Press Next at the bottom of the page.

Third Page of the Installation Wizard

The third step of the process is where you provide the details you will find in the Installation Checklist.



Please enter

- **Cluster Name**: This is an Azure Kubernetes Service (AKS) cluster name of the Kubernetes cluster to be created. The default, if used, is k2v-aks.
- **Environment Tag**: This tag describes the project's environment. Typical values could be Dev, UAT, and Prod. Azure cost management modules will use this tag to help you categorize costs.
- **Project Tag**: This tag describes the functional purpose of the project. You might consider tag names such as K2view TDM, K2view Masking, and K2view 360. Azure cost management modules will use this tag to help you categorize costs.
- **Domain Ingress**: This is the FQDN of the Nginx Ingress Controller for which a wildcard certificate needs to be created. In our example, we used acm.cloud.k2view.com, a domain owned by K2view. You will need to select your own domain and one for which you can create TLS certificates (PEM encoded).
- **TK2view Cloud Mailbox ID** provided to you by K2view
- **The base64-encoded values of your PEM TLS certificate and its associated private key**

  Your network team must create a wildcard TLS certificate for the domain you selected as your Domain Ingress. The full certificate authority chain needs to be present. During the installation, you will need to paste the base64-encoded values of your TLS certificate and its associated private key.

  - **TLS Private**: the base64-encoded private key associated with the TLS certificate
  - **TLS Certificate**: a base64-encoded wildcard certificate. For example, for the domain ingress of acm.cloud.k2view.com you need to create a wildcard certificate for *.acm.cloud.k2view.com for hosts of this domain.

What you paste in should look like the following:

base64-encoded TLS Private Key:

LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0tCk1JSUpRd0lCQURBTkJna3Foa2lHOXcwQkFRRUZBQVNDQ1
Mwd2dna3BBZ0VBQW9JQ0FRRE9yakUrN0NQVkU2QVgKV0lFRFVvdnVLc0lXOEJ4MTR
L…
QTlZV1pKKzlqRkhFUTdBZXB3S1RtMEVraz0KLS0tLS1FTkQgUFJJVkFURSBLRVktLS0tLQo=

base64-encoded TLS Certificate:

LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUdGRENDQlB5Z0F3SUJBZ0lTQS9YVFJKaUpXOXMzbl
dZZlN5a0RpZlh5TUEwR0NTcUdTSWIzRFFFQkN3VUEKTURNeEN6QUpCZ05WQkFZVEFsVlRNUll3RkFZRFZR
UUtFdzFNWlhRbm
…
1ND0KLS0tLS1FTkQgQ0VSVElGSUNBVEUtLS0tLQ==

Note: The PEM certificate and key must be further encoded using base64 as input to the installation screen. To encode a PEM certificate (e.g., cert.pem) to base64 on both Linux and Windows, you can use the following commands.

Linux: You can use the base64 command to read the cert.pem file, encodes it in base64, and saves the output to cert_base64.txt.

```
base64 cert.pem > cert_base64.txt
```

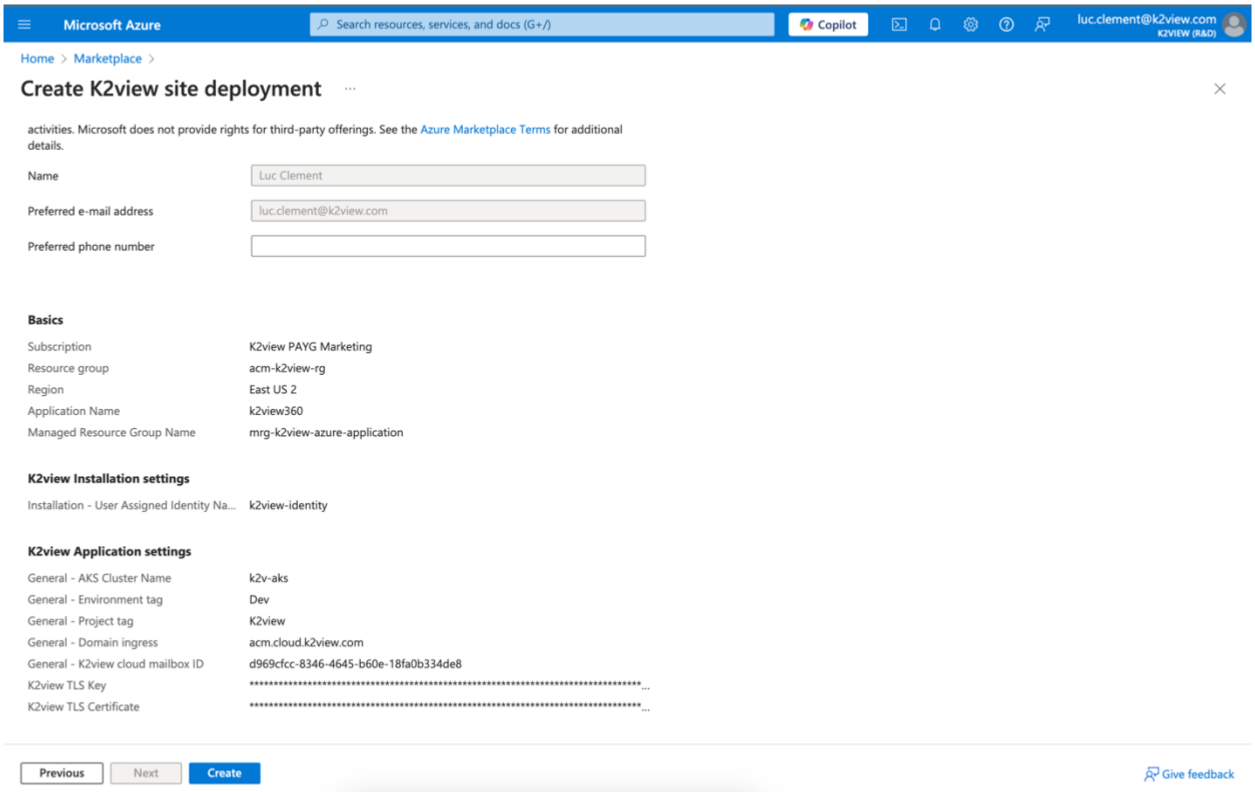
Windows: On Windows, you can use the certutil command to encode the cert.pem file in base64 and saves the result to cert_base64.txt.

```
certutil -encode cert.pem cert_base64.txt
```

Prerequisite: You should prepare these base64 encoded files before the start of the installation.

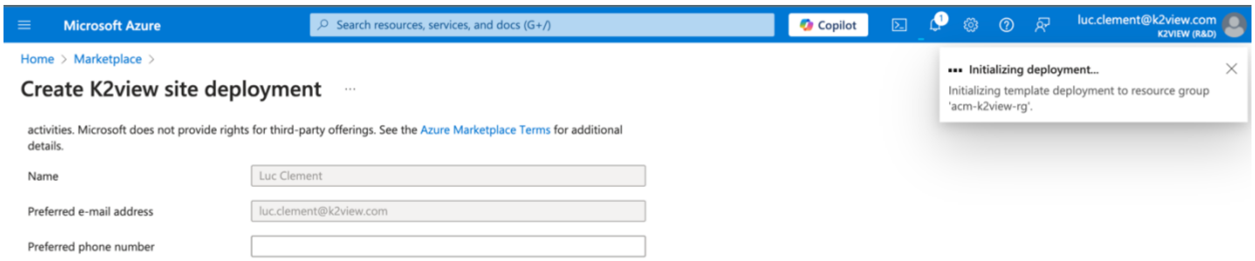
Press Next at the bottom of the page.

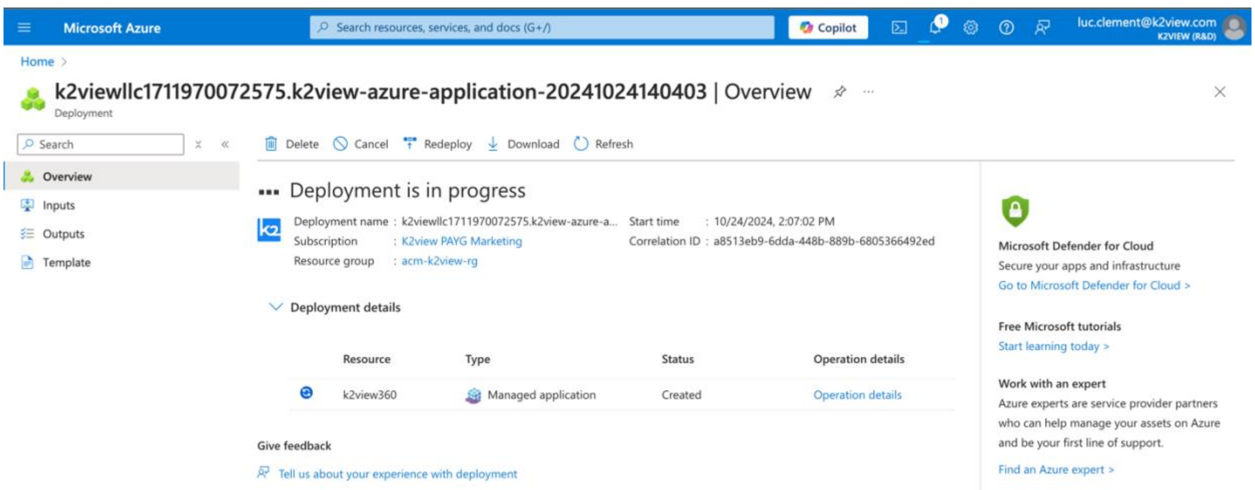
After you press Next, this page will be shown. When you are ready, scroll down and press Create.

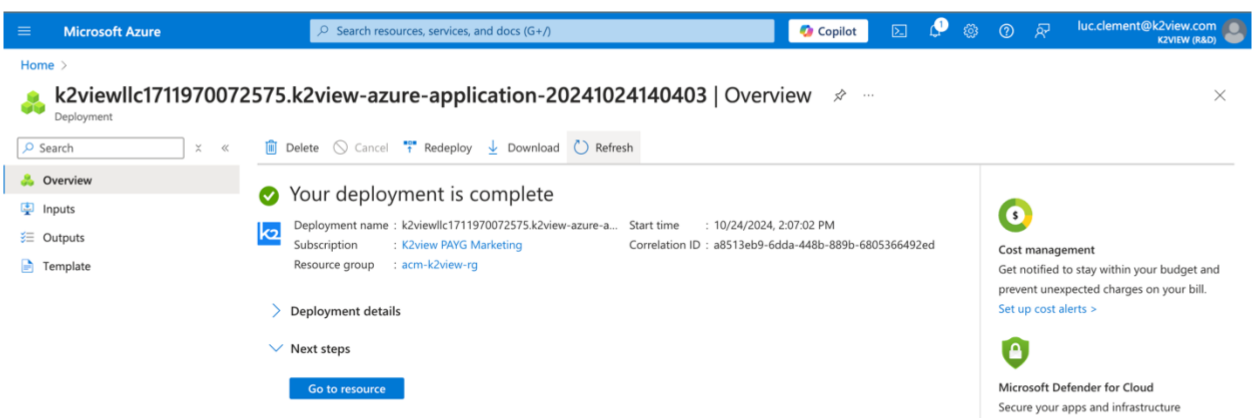


The Installation Process Begins

It will take approximately 15 minutes to create all resources in the **Azure Application Resource Group** And **Azure Managed Application Resource Group** you selected on the first page of the installation wizard.



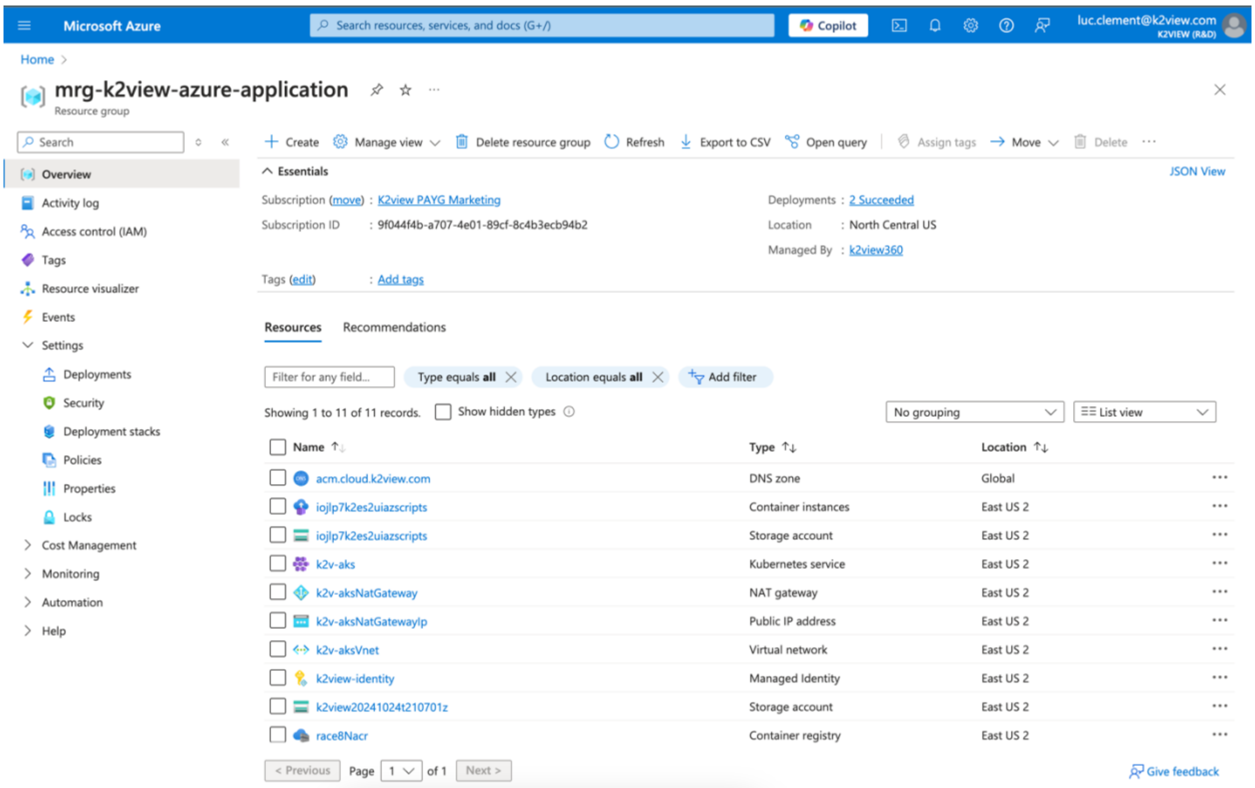




The Azuse Application installation has been completed.

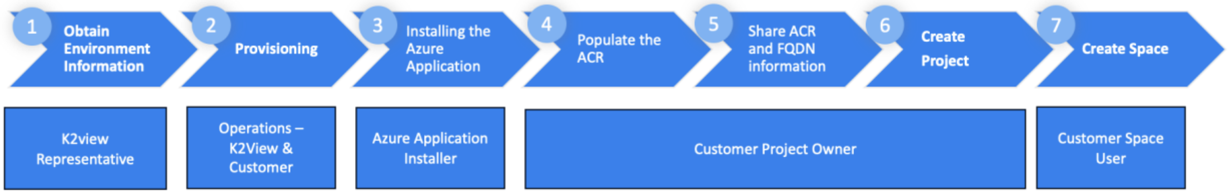
You will find the following created resources by navigating to the newly created resource group.

A Terraform module is used behind the scenes to take these input parameters to deploy K2View Fabric services in your environment. For more information on the details and configurations performed by the Azure Application installer, you can review the Terraform module available at: https://github.com/k2view/blueprints/tree/main/Azure/terraform/AKS.



You are now ready to proceed to the next step: populate the Azure Container Registry created for you with K2view service Docker images.

4. Populating Your Azure Container Registry



To populate your Azure Container Repository, you will need

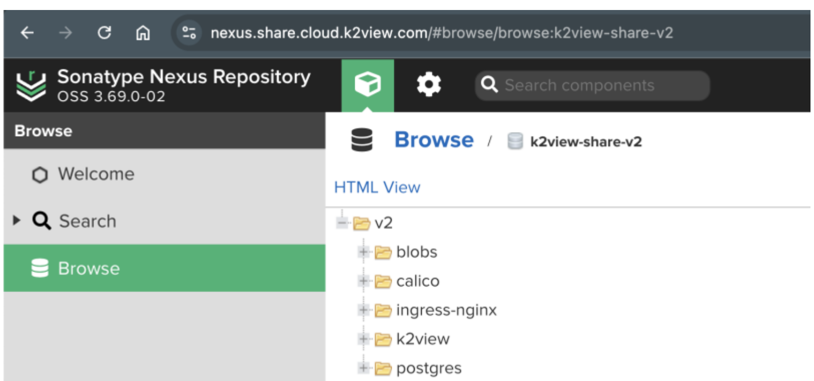
1. K2view Nexus Repository Account supplied to you by K2view
2. Obtained the list of Docker images to pull from Nexus and push to in consultation with K2view
3. Docker installed on the computer used to pull and push Docker images to your ACR

The Planning section describes what is necessary to be ready to perform this step.

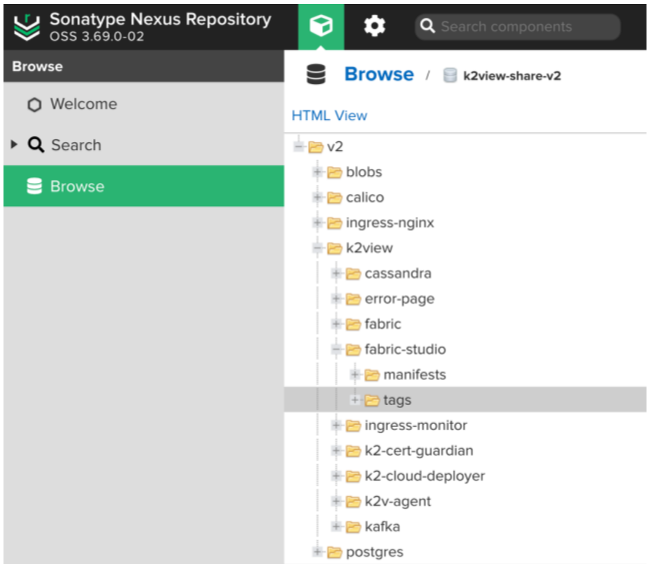
4.1. Getting Acquainted with the K2view Nexus Repository

To acquaint yourself of the K2view Docker images available to you can you login to the K2view Nexus web application. Here you can find the versions available to you. Please consult with your K2view representative which images and versions you should use. We recommend using the latest version of the K2view Docker images.

Login and navigate to https://nexus.share.cloud.k2view.com/#browse/browse:k2view-share-v2 to view available images



For example, you can find the latest version of K2view's Fabric Studio by expanding the tags folder.



4.2. Commands to Pull Containers from the K2view Nexus Repository

The list of K2view service Docker images will be discussed and recommended to you in consultation with your K2view representative.

For the purpose of these instructions we will "pull" from the K2view Nexus repository three images of a specific version: fabric-studio, fabric, and the k2-cloud-deployer Docker images.

Docker tools need to be installed on the computer you are performing this installation. You can download these from https://docker.com.

First Login

Command:

`docker login -u [your K2view Nexus username]` [`https://docker.share.cloud.k2view.com`](https://docker.share.cloud.k2view.com)

Second, pull the required images. In this example we pull three.

Commands: In succession, please run the following three commands. Keep in mind that the version and docker images needed may differ.

```
docker pull docker.share.cloud.k2view.com/k2view/fabric-studio:8.1.0_199
docker pull docker.share.cloud.k2view.com/k2view/fabric:8.1.0_199
docker pull docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:1.8.0
```

4.3. Commands to Push Containers to the Azure Container Registry

Now that you have “pulled” images from the K2view Nexus repository, you are now ready to “push” these to the Azure Container Registry (ACR) created for you. If you wish to use a preexisting ACR or OCI-compliant container registry, please also consult the [Selecting a Different Azure Container Registry or OCI-Compliant Registry]() section.

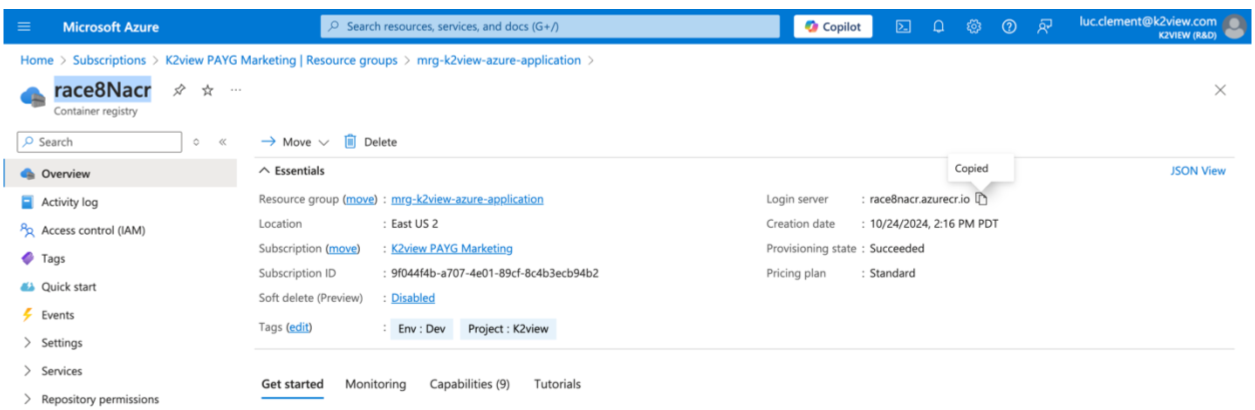
First, obtain the credentials required to perform a Docker Push to your ACR

Installing the K2view Azure Application created an ACR within the Azure Managed Application Resource Group**.** You can obtain the credential associated with it by navigating you the resource group and selecting the ACR created for you.

In this example, the Managed Application Resource Group is named mgr-k2view-azure-application, and the created ACR is named rac8Nacr (a random name).

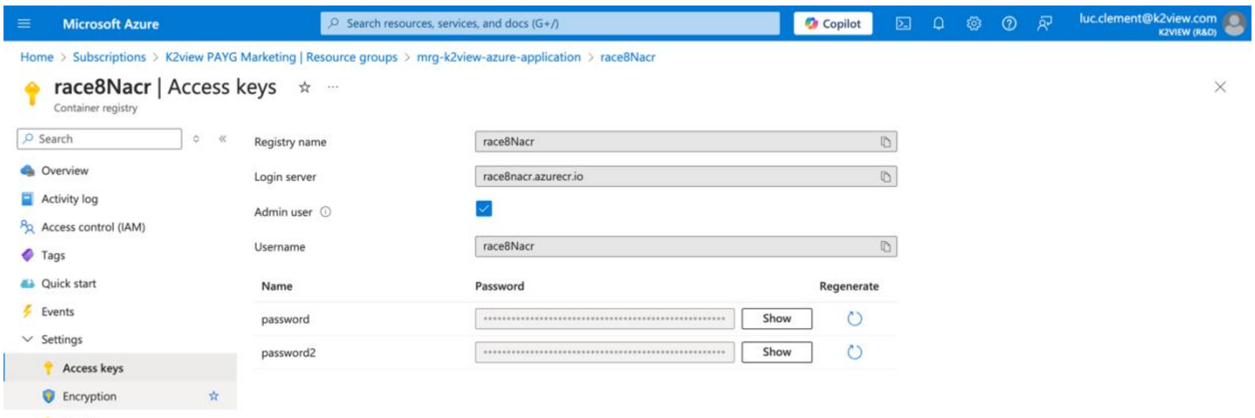
When you navigate to this ACR location, please

1. Get the URL of the login server, which is shown here. Copy the URL which will be required for your Docker login and push commands.



2. Select Access Keys in the left navigation menu under Settings to obtain your credentials. From this page:
    a. Copy the username
    b. Use the Show button to obtain the password, and copy this password

    This information will be required to perform the login to your ACR using a Docker command



Second, perform the Docker Push to your ACR

Use the Docker login command to login to your ACR

`docker login -u [your ACR username] [the ACR Login server]`

In our example we used: `docker login -u race8Nacr race8nacr.azurecr.io`

Third, tag each image you pulled from the K2view Nexus

Use of tags is for version control. Tagging Docker images allows you to maintain different versions of the same application, making it easier to roll back to a previous version if needed. This can be especially useful in cases where a new release introduces a bug or breaks compatibility with other components.

In our example, we used the following three commands, respectively, for the fabric-studio, fabric, and k2-cloud-deployer Docker images. Note that the highlighted `race8nacr` value is the name used in our example, you will need to use the one associated with your ACR.

```
docker tag docker.share.cloud.k2view.com/k2view/fabric-studio:8.1.0_199
race8nacr.azurecr.io/k2view/fabric-studio:8.1.0_199
```

```
docker tag docker.share.cloud.k2view.com/k2view/fabric:8.1.0_199
race8nacr.azurecr.io/k2view/fabric:8.1.0_199
```

```
docker tag docker.share.cloud.k2view.com/k2view/k2-cloud-deployer:8.1.0_199
race8nacr.azurecr.io/k2view/k2-cloud-deployer:8.1.0_199
```

Fourth, push images to your ACR

Next, you need to push the Docker images you pulled from the K2view Nexus to your ACR using the command: `docker push [image location]`.

In our example, we used:

```
docker push race8nacr.azurecr.io/k2view/fabric-studio:8.1.0_199

docker push race8nacr.azurecr.io/k2view/fabric:8.1.0_199

docker push race8nacr.azurecr.io/k2view/k2-cloud-deployer:1.8.0
```

Fifth, please provide the locations you used to K2view

K2view needs these ACR locations of your images to finalize your site's configuration. Please note that you will share these locations with K2view as described in the Sharing Information with K2view section.

4.4. Selecting a Different Azure Container Registry or OCI-Compliant Registry

You must follow similar steps if you choose to use a pre-existing ACR or an OCI-compliant registry. You will also need to provide K2view with the locations of your registry.

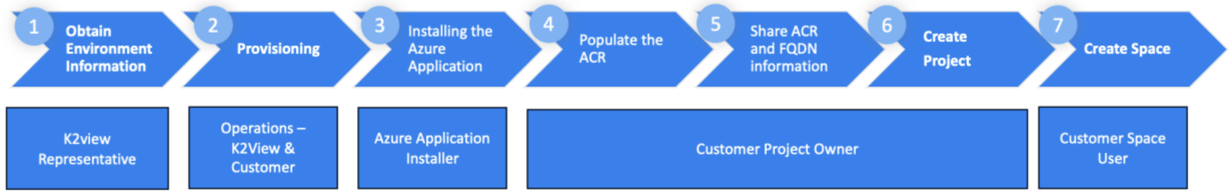
4.5. Sharing Information with K2view

The Customer Project Owner must share the following information to complete the configuration for the tenant's creation of Projects and Spaces.

- The fully qualified domain name (FQDN) used for your Domain Ingress.
    - For example, acm.cloud.k2view.com was used. You need to use a domain your organization owns.
- The list of Azure Container Registry (ACR) locations of your Docker images
    - E.g., the locations shown in the fourth step of the Commands to Push Containers to the Azure Container Registry section.
- The AKS Cluster name

This information may have been identified during pre-planning, but the specific information should be confirmed to avoid misconfiguration.

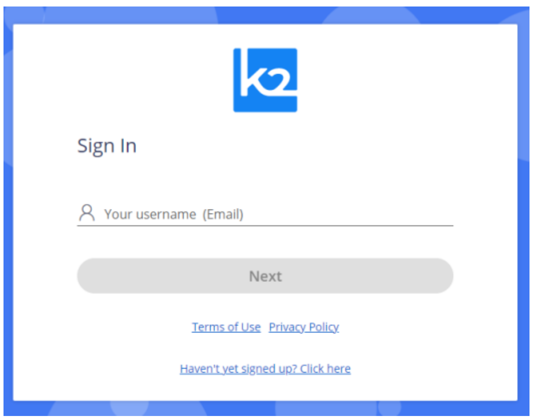
5. Creating Your Fabric Project



5.1. Sign In to K2cloud

To sign in, access **cloud.k2view.com** using Chrome, Microsoft Edge, or Firefox web browsers.

When prompted to sign in, enter your email address. This account needs to have been provisioned for you by K2view. Please refer to the Prerequisites section.



If provisioned by K2view, you will receive an access code via email you will then enter in the prompt window that is shown when you press Next.

You will be directed to the Spaces page, where you will eventually create spaces. First, you need to create a project.

5.2. Create the Project

You create a project by selecting the Projects top-level navigation tab and pressing “Create Project”. A project requires a name and an associated Git repository. You must provide Git-related information to create a project. To learn more about Git and how to obtain the Repository URL you will use, a branch, and your token, please review the GitHub section.

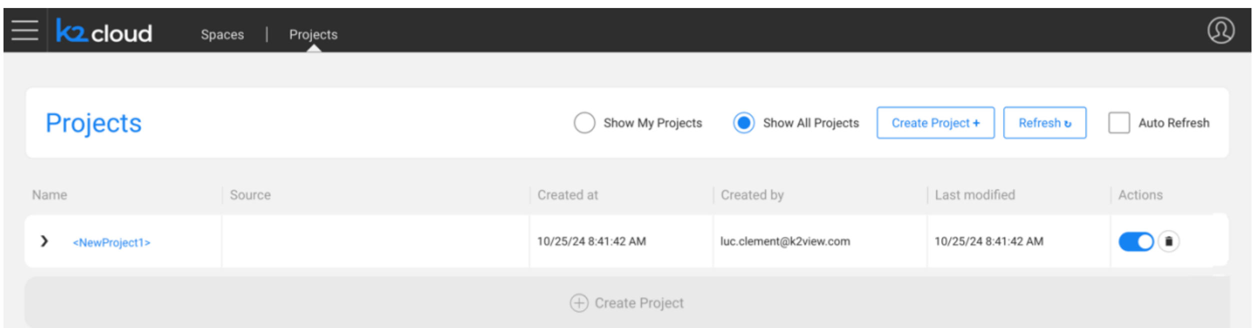
The Fabric spaces you implement are based on a “Space Profile”. You can configure more than one Space Profile per Project. If you need assistance, a K2view representative will assist you. Some of this information will have been shared with you during the Planning phase.

To create a Space Profile, you need to:

- Give it a name that will have significance to your users
- Select an Image Profile identified in consultation with K2view, and
- Define the site where this space will operate within. K2view will help you define this list.

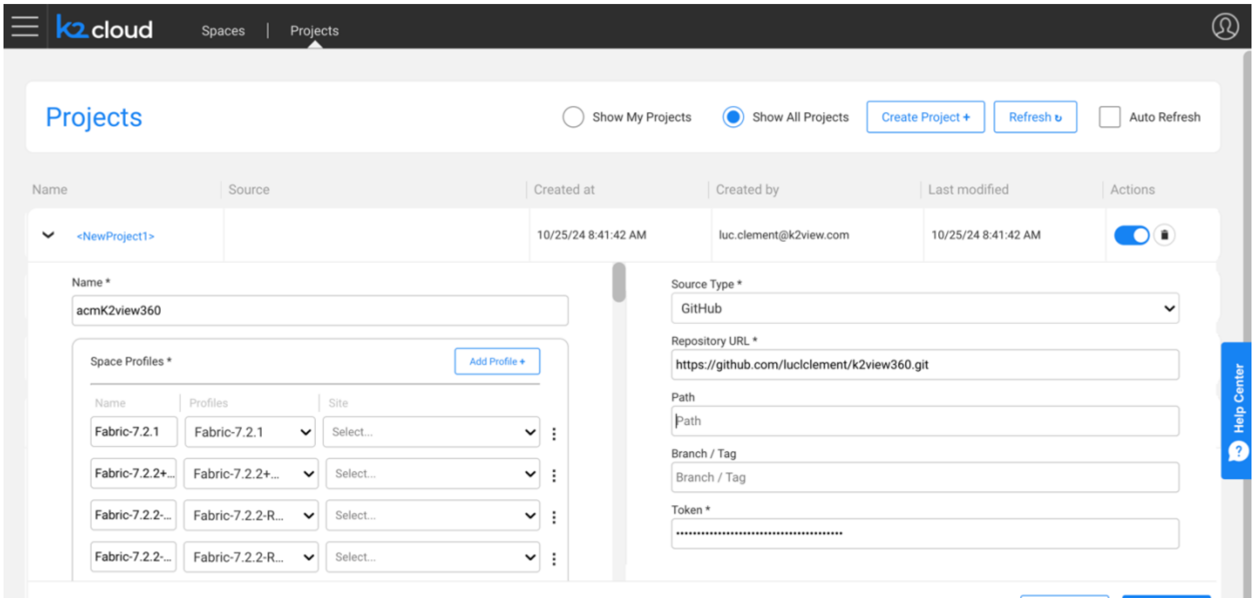
Creating a Project

After pressing Create Project, an empty project will be created.

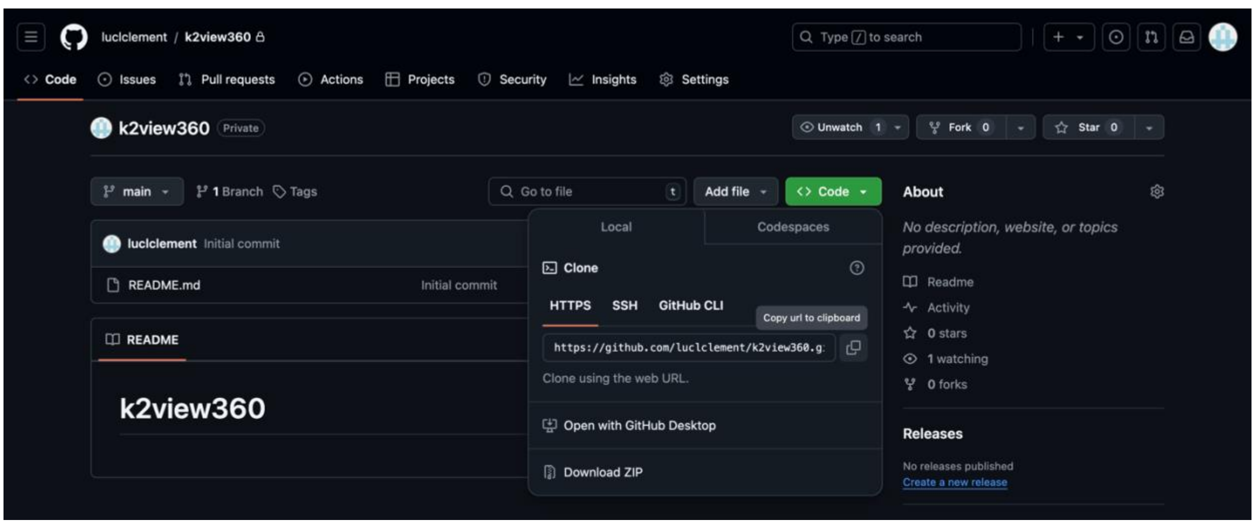


Provide the project with a name and select your space profiles. Again, K2view will have provided this.

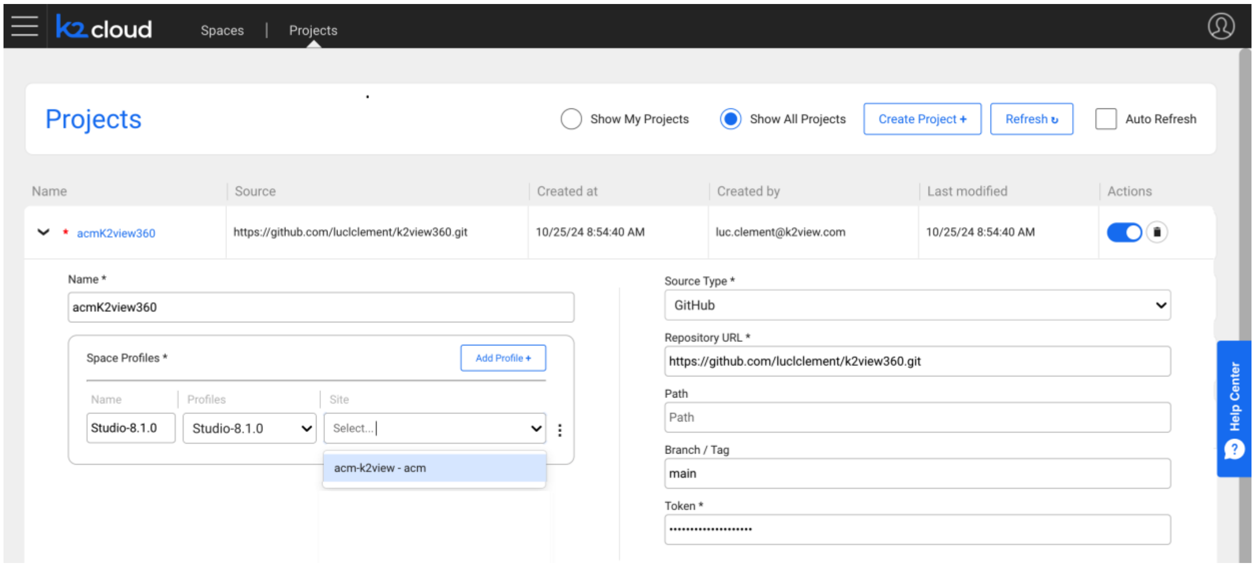
You also need to provide the details of your Git source control system described in the GitHub section.



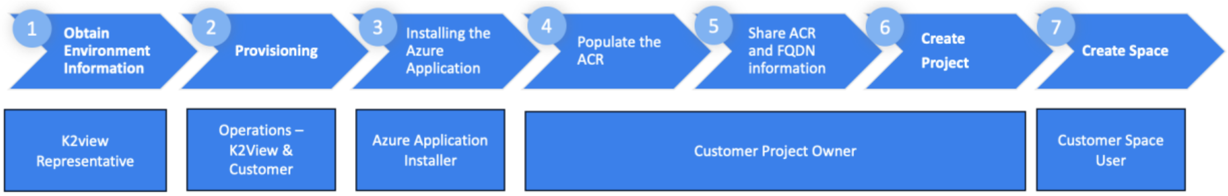
For example, you can obtain the URL from your Git repository, as shown here in our example.



And, finally, please select the site created from the site's dropdown.



6. Creating Your First Fabric Space



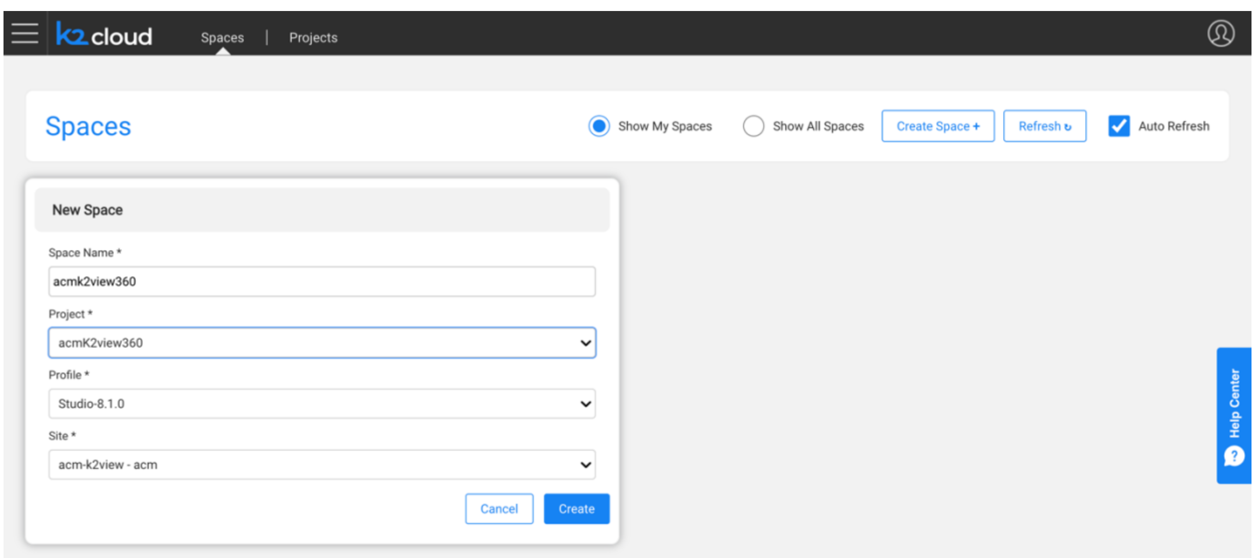
After your Project is created, you can create a Fabric Space. Here, we show an example of an existing space and a new space being defined.

To create a space

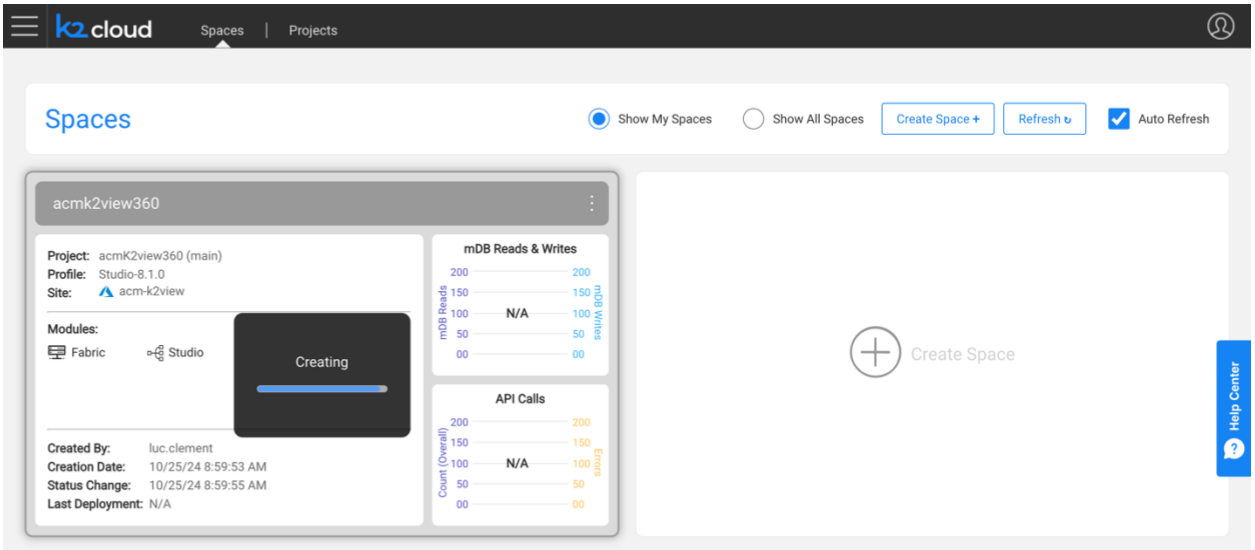
- Give it a name,
- Select the project you created,
- Select your intended profile, one recommended by K2view given your intended purpose, and
- Select site.

If you need to install a K2view extension such as TDM, you must go to your space and install the extension from K2exchange.

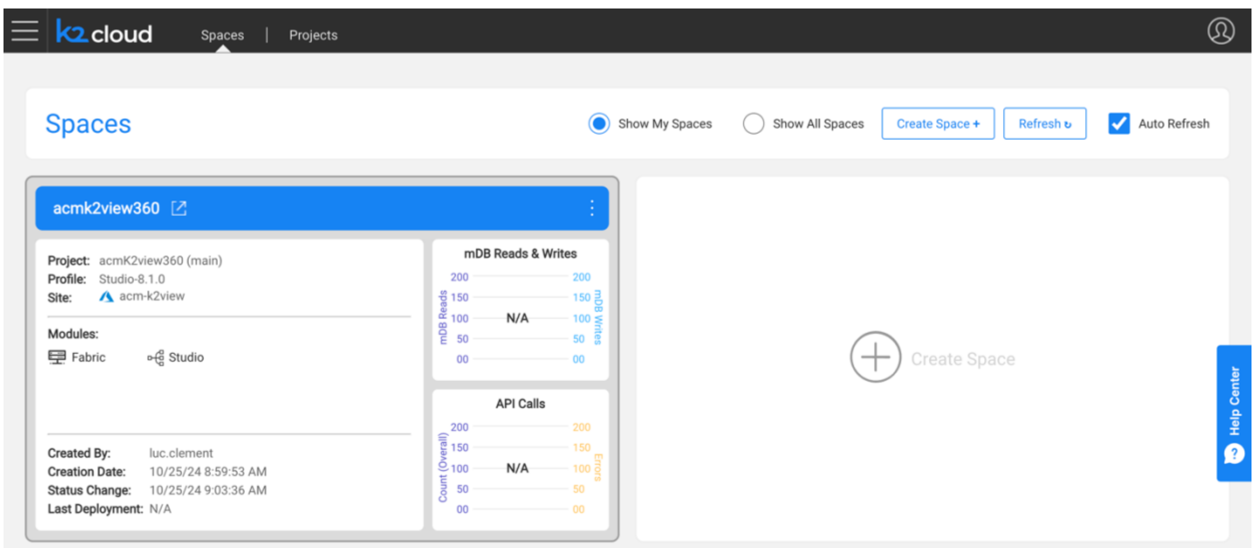
When ready, press Create to trigger the space's creation.



The Space creation process begins and may take a few minutes to complete.



Once complete your site is displayed to you.



You have successfully created your Project and Space and are ready to begin working with K2view.

7. Validation

You are now ready to proceed with validating your space. Doing so is performed by selecting the top right arrow icon (see next section).

This will cause the browser to connect to the Space you created over HTTPS to the environment you created. The TLS certificate you assigned will be used for this connection.

DNS

During the installation, a DNS zone was created. This zone may take time to propagate into your environment and may not be available until it does.

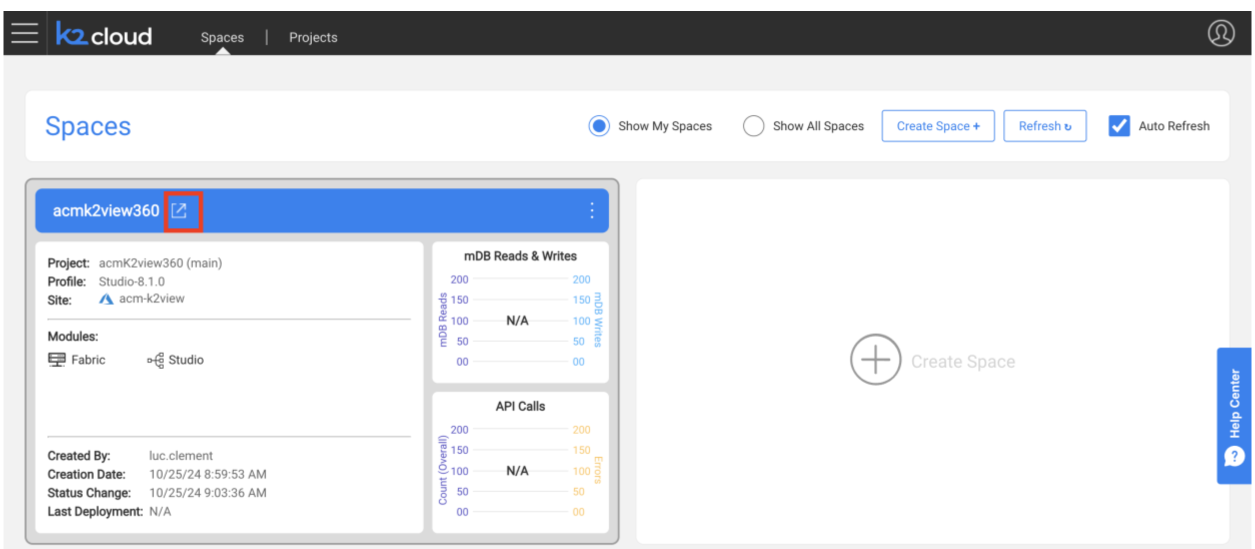
You should note that when creating a namespace, its name is associated as a subdomain to this domain name in the Ingress controller. For example, if the domain is "k2dev.company.com" and a created namespace is "test", then the URL of this namespace that users shall access will be "test.k2dev.company.com".

In our example, we will connect to a space accessible at acmk2view360-k2v-rnd.acm.cloud.k2view.com, a publicly resolvable domain.

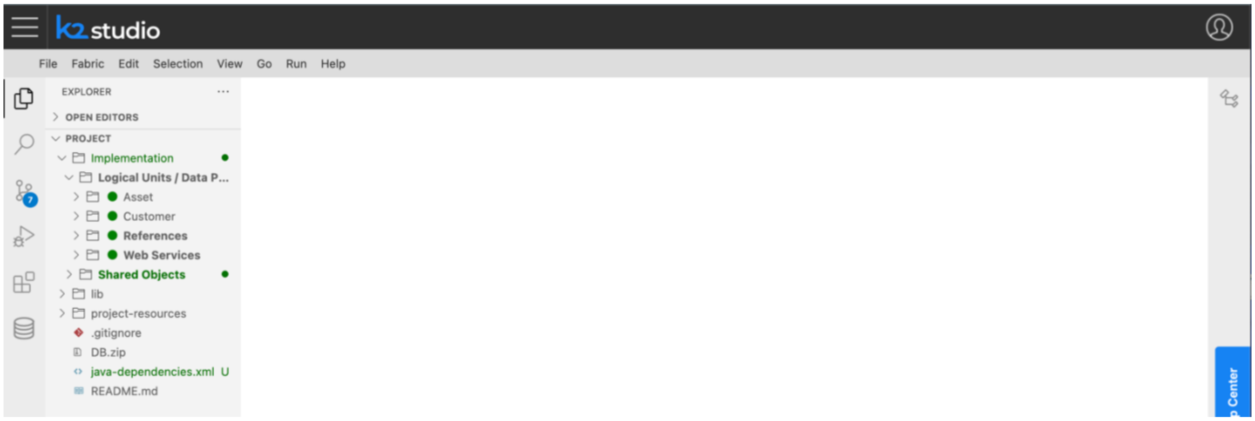
Note: Some organizations may not allow the use of DNS records created in your installation's Public DNS zone to propagate your organization's DNS. If that's the case, you must provide the address information to your organization. Refer to the DNS zone section for this.

7.1. Enter The Fabric Space

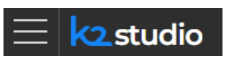
To enter your space, select the top left corner to launch the workspace.



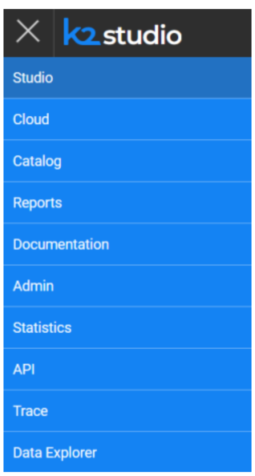
The **Studio** web application will be displayed.



Then, navigate to the various applications' context menus and confirm that each web application appears functional by selecting the “hamburger” menu item on the top left of the page.



The following is the list of web applications you should expect to see.



8. Completion

This completes the end of your installation and configuration of the K2view Azure Application, the creation of your K2view Project and the creation of your first K2view Project.

We invite you to visit our Knowledge Base at https://support.k2view.com/knowledge-base.html, where you can find documentation and demo projects, and sign up for the K2view Academy Learning Center at https://academy.k2view.com. For access, please talk to your K2view representative.

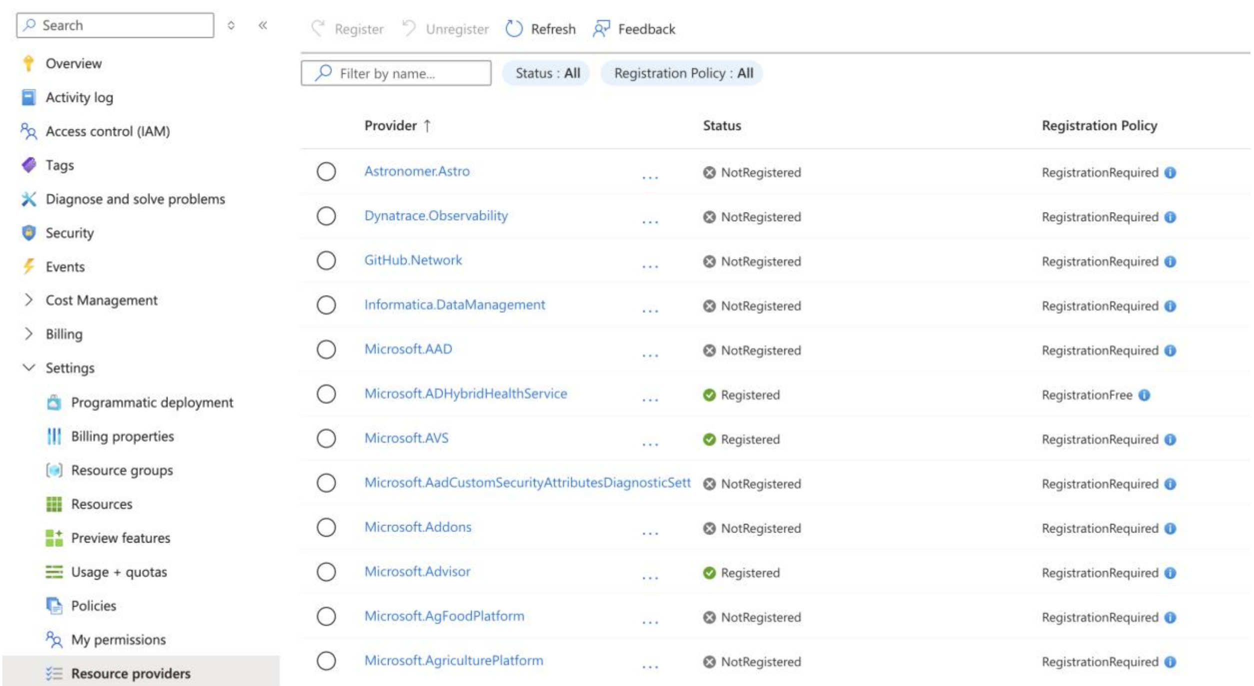
9. Instructions

9.1. Azure Resource Registration

The List of Azure Resources

An Azure resource provider is a set of REST operations that enable functionality for a specific Azure service. When deploying K2view Fabric services within an Azure environment, resource providers must be registered before the start of the installation.

The act of registering resources is a privileged action that needs to be performed by a Subscription Owner. This can be performed using the Azure UI at the subscription level as shown here, or automated using the Azure CLI (described in the next section).



The following list of Azure Resources required by K2view Fabric services need to be registered at the Subscription level.

- ManagedIdentity
- ContainerService
- Storage
- ContainerRegistry
- Maintenance
- CognitiveServices
- HDInsight
- Media
- ServiceBus
- SignalRService

- Search
- AppConfiguration
- SecurityInsights
- Maps
- DataLakeAnalytics
- Kusto
- Databricks
- StreamAnalytics
- DocumentDB
- MachineLearningServices
- EventGrid
- insights
- DBforPostgreSQL
- DBforMySQL
- OperationsManagement
- OperationalInsights
- Logic
- Compute
- ServiceFabric
- AppPlatform
- DesktopVirtualization
- PowerBIDedicated
- Cache
- GuestConfiguration
- BotService
- AVS
- DevTestLab
- DataMigration
- Cdn
- Web
- EventHub
- Security
- Network
- Automation
- Devices
- DBforMariaDB
- PolicyInsights
- ManagedServices
- Relay
- DataProtection
- CustomProviders

- Sql
- HealthcareApis
- Management
- DataFactory
- RecoveryServices
- MixedReality
- ApiManagement
- KeyVault
- NotificationHubs
- DataLakeStore

Automating Resource Registration Using the Azure CLI

The Azure CLI automates the above. Once installed and login using the “az login” command, you can paste the following within the Azure CLI window.

After a few minutes all required resources will have been registered. You can use the Azure UI (shown above) to track progress and a CLI command (az provider list --query="[?registrationState=='Registered']" --output table) to do so.

```
az provider register --namespace Microsoft.ManagedIdentity
az provider register --namespace Microsoft.ContainerService
az provider register --namespace Microsoft.Storage
az provider register --namespace Microsoft.ContainerRegistry
az provider register --namespace Microsoft.Maintenance
az provider register --namespace Microsoft.CognitiveServices
az provider register --namespace Microsoft.HDInsight
az provider register --namespace Microsoft.Media
az provider register --namespace Microsoft.ServiceBus
az provider register --namespace Microsoft.SignalRService
az provider register --namespace Microsoft.Search
az provider register --namespace Microsoft.AppConfiguration
az provider register --namespace Microsoft.SecurityInsights
az provider register --namespace Microsoft.Maps
az provider register --namespace Microsoft.DataLakeAnalytics
az provider register --namespace Microsoft.Kusto
az provider register --namespace Microsoft.Databricks
az provider register --namespace Microsoft.StreamAnalytics
az provider register --namespace Microsoft.DocumentDB
az provider register --namespace Microsoft.MachineLearningServices
az provider register --namespace Microsoft.EventGrid
az provider register --namespace microsoft.insights
az provider register --namespace Microsoft.DBforPostgreSQL
az provider register --namespace Microsoft.DBforMySQL
az provider register --namespace Microsoft.OperationsManagement
az provider register --namespace Microsoft.OperationalInsights
az provider register --namespace Microsoft.Logic
az provider register --namespace Microsoft.Compute
az provider register --namespace Microsoft.ServiceFabric
az provider register --namespace Microsoft.AppPlatform
```



```
az provider register --namespace Microsoft.DesktopVirtualization
az provider register --namespace Microsoft.PowerBIDedicated
az provider register --namespace Microsoft.Cache
az provider register --namespace Microsoft.GuestConfiguration
az provider register --namespace Microsoft.BotService
az provider register --namespace Microsoft.AVS
az provider register --namespace Microsoft.DevTestLab
az provider register --namespace Microsoft.DataMigration
az provider register --namespace Microsoft.Cdn
az provider register --namespace Microsoft.Web
az provider register --namespace Microsoft.EventHub
az provider register --namespace Microsoft.Security
az provider register --namespace Microsoft.Network
az provider register --namespace Microsoft.Automation
az provider register --namespace Microsoft.Devices
az provider register --namespace Microsoft.DBforMariaDB
az provider register --namespace Microsoft.PolicyInsights
az provider register --namespace Microsoft.ManagedServices
az provider register --namespace Microsoft.Relay
az provider register --namespace Microsoft.DataProtection
az provider register --namespace Microsoft.CustomProviders
az provider register --namespace Microsoft.Sql
az provider register --namespace Microsoft.HealthcareApis
az provider register --namespace Microsoft.Management
az provider register --namespace Microsoft.DataFactory
az provider register --namespace Microsoft.RecoveryServices
az provider register --namespace Microsoft.MixedReality
az provider register --namespace Microsoft.ApiManagement
az provider register --namespace Microsoft.KeyVault
az provider register --namespace Microsoft.NotificationHubs
az provider register --namespace Microsoft.DataLakeStore
```

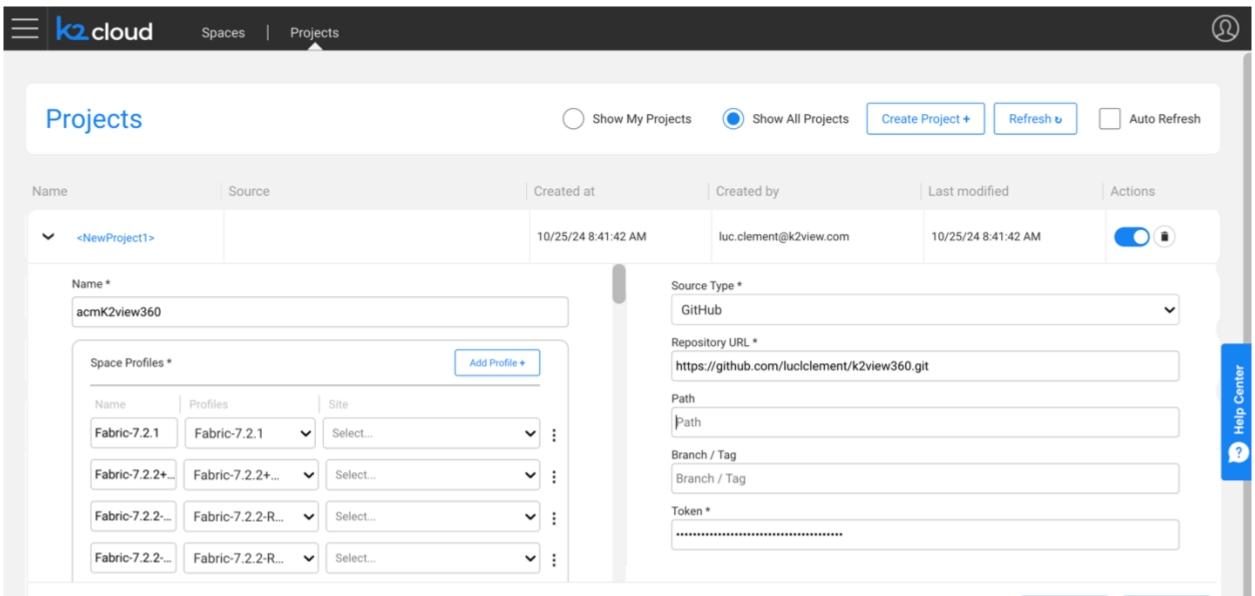
9.2. GitHub

Your organization may already have a Git repository. GitHub, GitLab and BitBucket are cloud providers that you can turn to should you like to get started.

During the course of this installation you will be asked to provide Git source code repository information including:

1. The provider: GitHub, GitLab, or BitBucket
2. A repository URL: in our example we used GitHub and this repository URL: https://github.com/luclclement/k2view360.git
3. A branch: to keep things simple we used "main"
4. A Git Token: to provide the project access to the Git account

This information is captured on the project's page

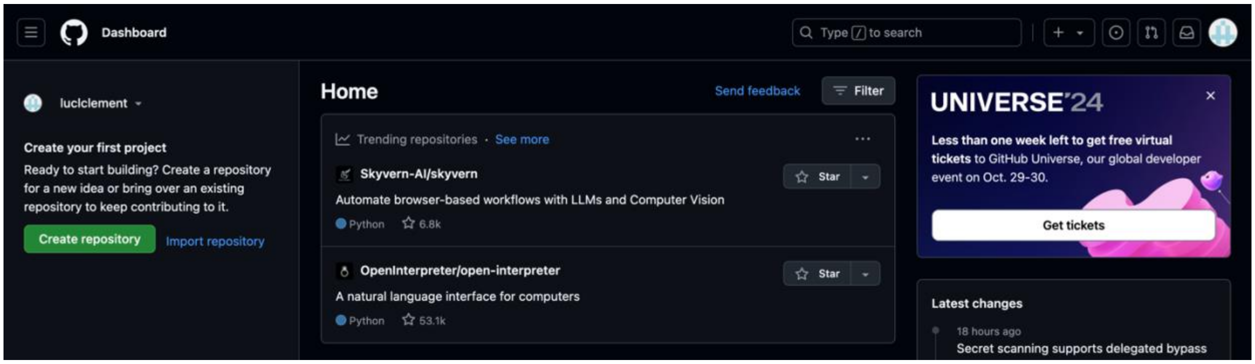


If you don't have one you can obtain from github.com. First you need to sign up at https://github.com

You will create a repository and then obtain a token.

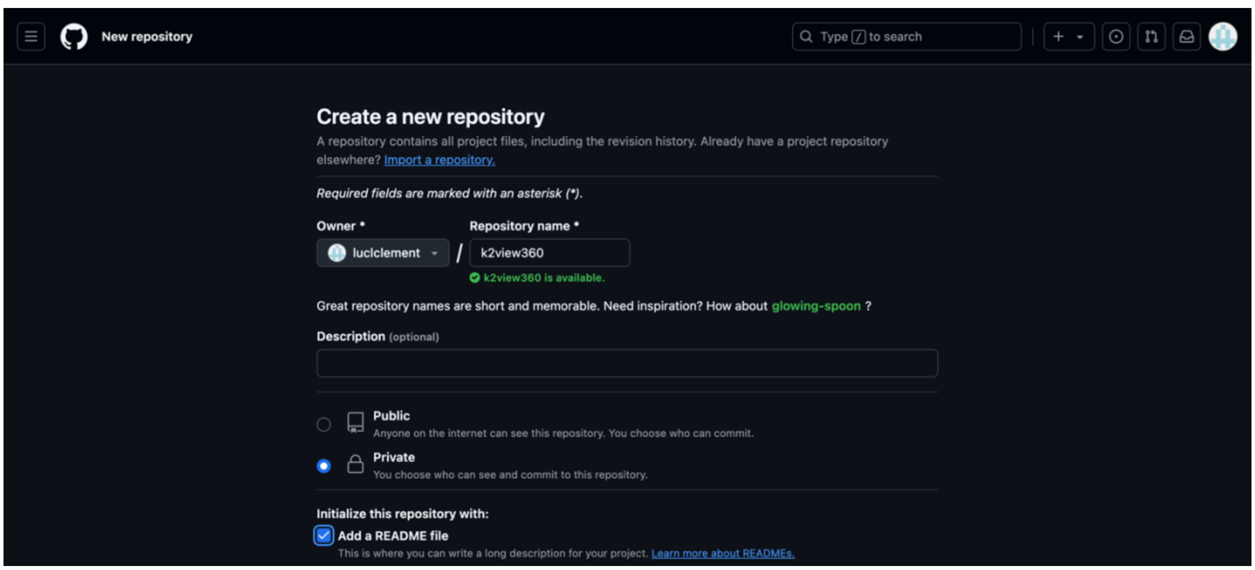
Create your repository

Click the “Create repository” button on the left side panel.

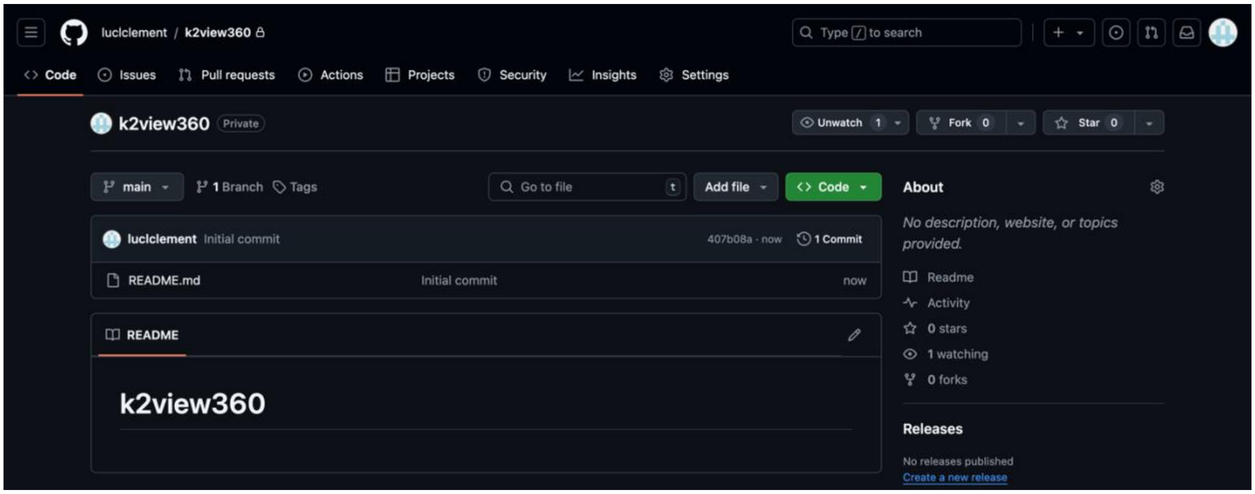


Give it a name. In this example we used k2view360.

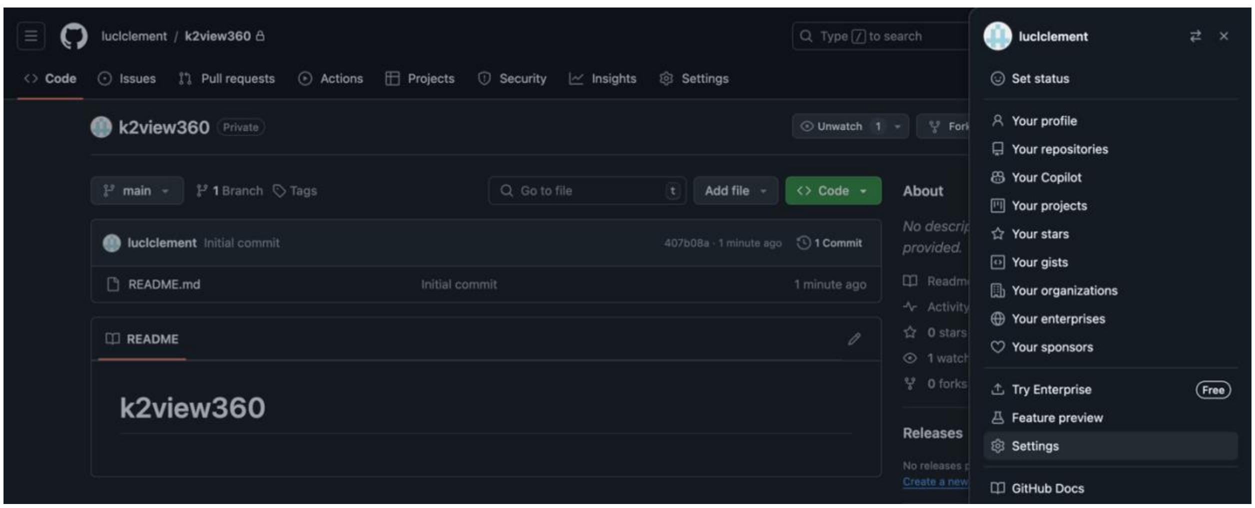
You should make it “Private”, and you should select the “Add a README file checkbox so that the repository is not empty.



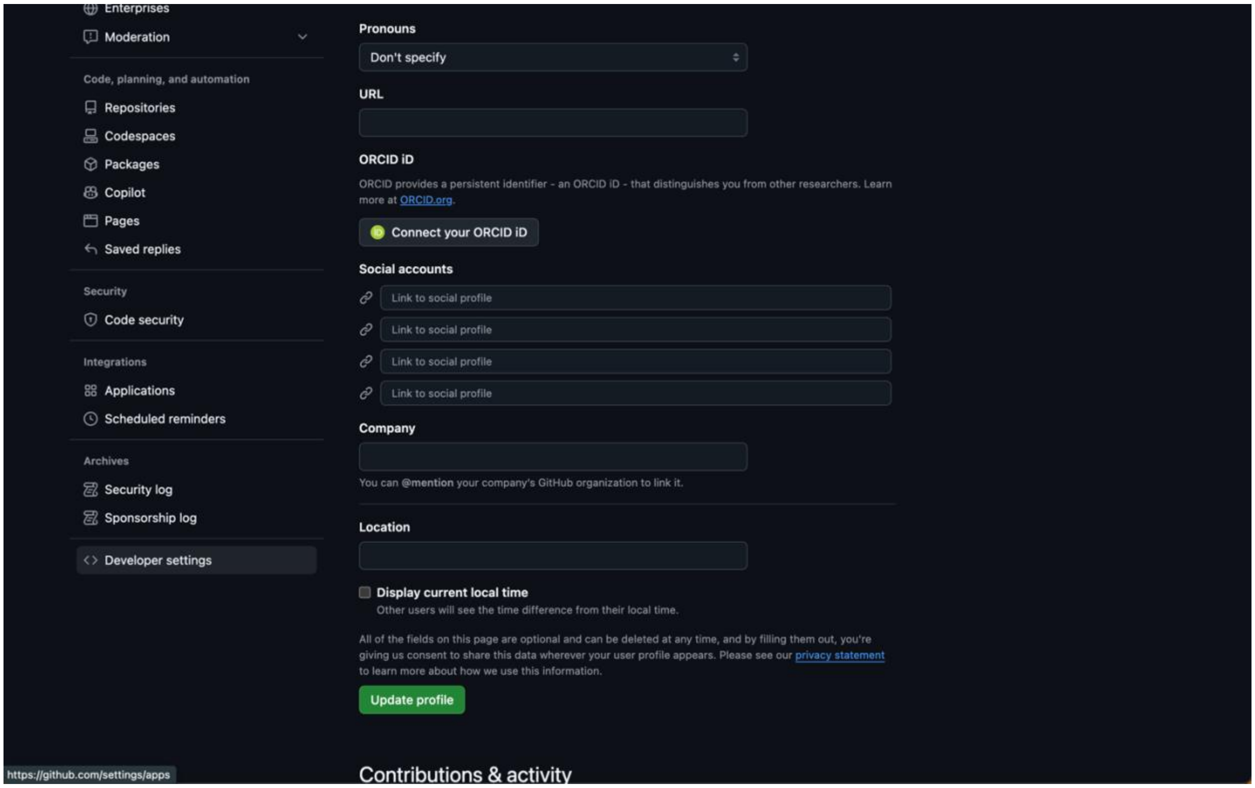
Once created you will find an empty README file.



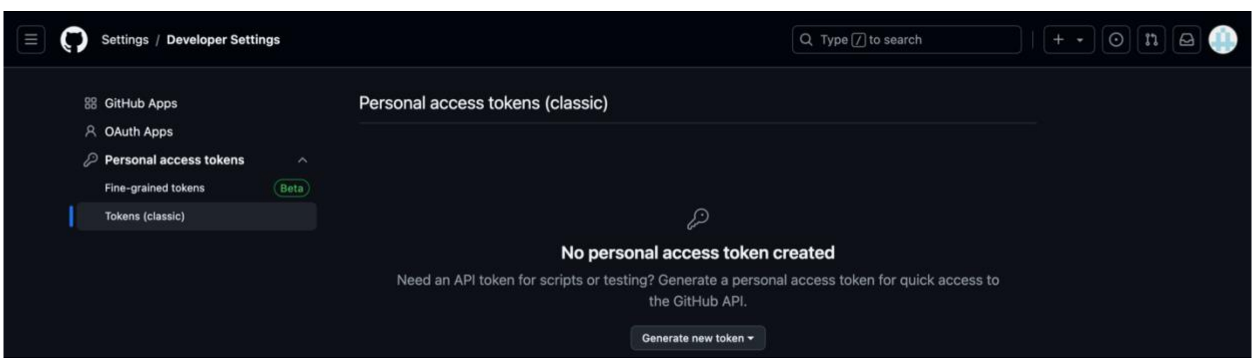
The next task is to obtain your Token. For this, drop down the user's icon on the top right of the page. Select Settings.



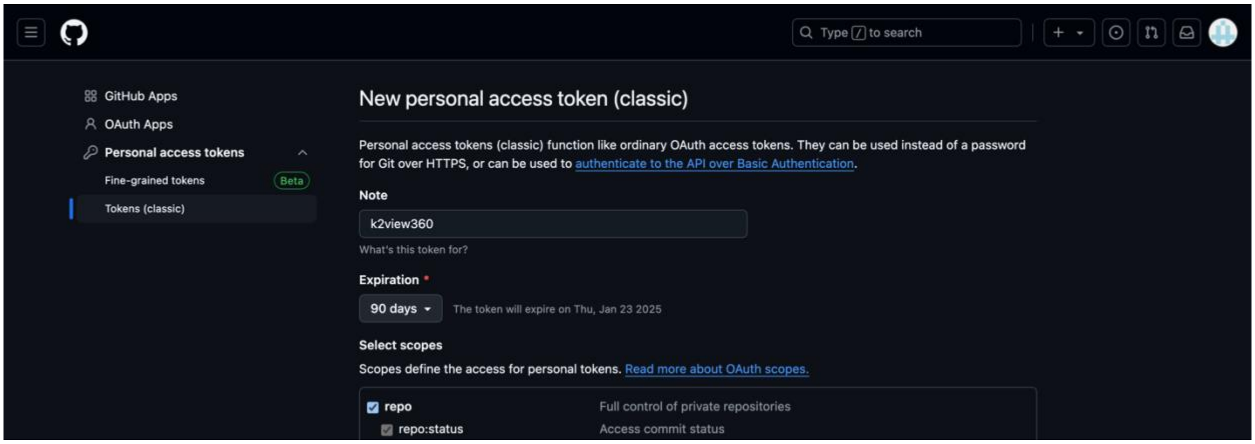
Then, from the left panel, select “Developer settings” on the bottom of the list.



Then, select “Personal access tokens” from the left panel, and select “Tokens (classic)” (if you’re adventurous, try the beta).



Select the “Generate new token” on the bottom of the page and create your token.



When the page refreshes you will be shown your token. Copy it to use in the Fabric Project creation page. Keep a copy of the token for safekeeping.

9.3. DNS zone

During the installation, a DNS zone will be created. This zone may take time to propagate into your environment and may not be available until it does.

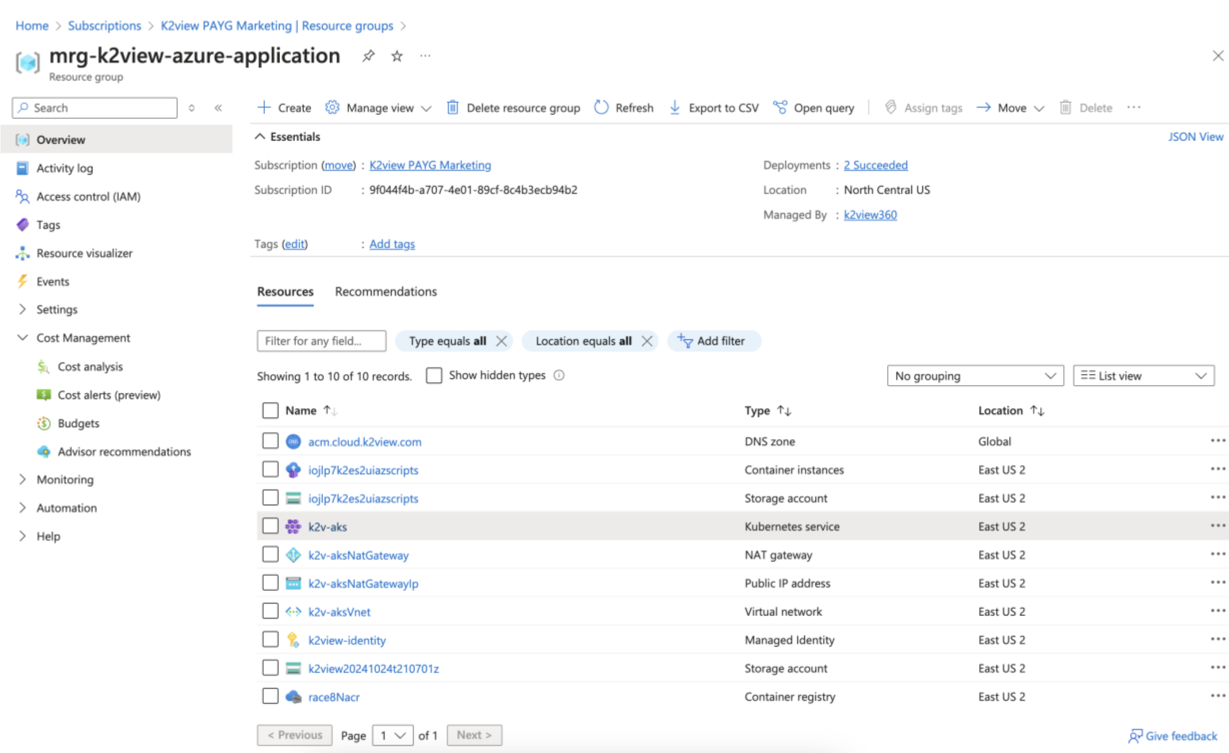
Some organizations may not allow the use of DNS record created in your installation's Public DNS zone to propagate outside the organization.

If that's the case, you will need to provide the address information to your organization to generate a DNS A record for the NGINX Ingress controller load balancer IP address created in our Public DNS Zone.

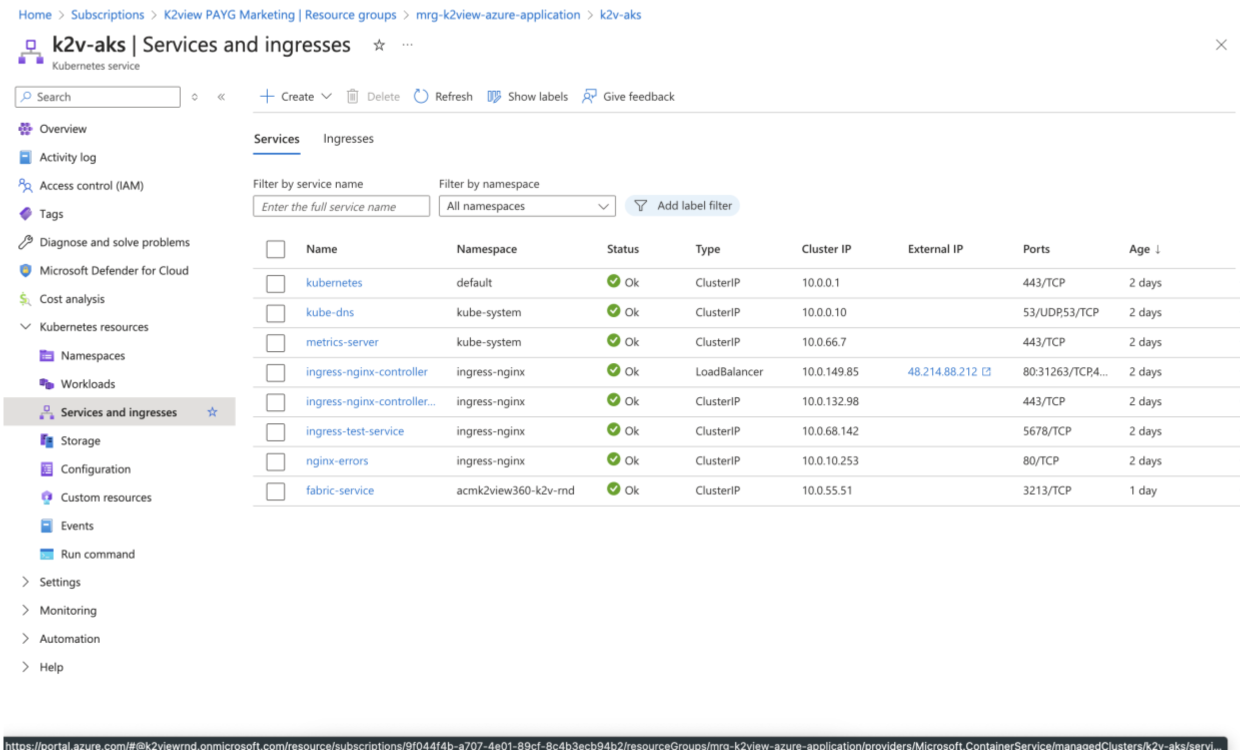
You should note that when creating a namespace, its name is associated as a subdomain to this domain name in the Ingress controller. For example, if the domain is "k2dev.company.com" and a created namespace is "test", then the URL of this namespace that users shall access will be "test.k2dev.company.com".

In our example, we will connect to a space accessible at acmk2view360-k2v-rnd.acm.cloud.k2view.com, a publicly resolvable domain.

To locate the address, select the Kubernetes service



Select the Services and ingresses. The address you are looking for is the ingress-nginx-controller's load balancer. In this example the address is 48.214.88.212.



9.4. Certificates

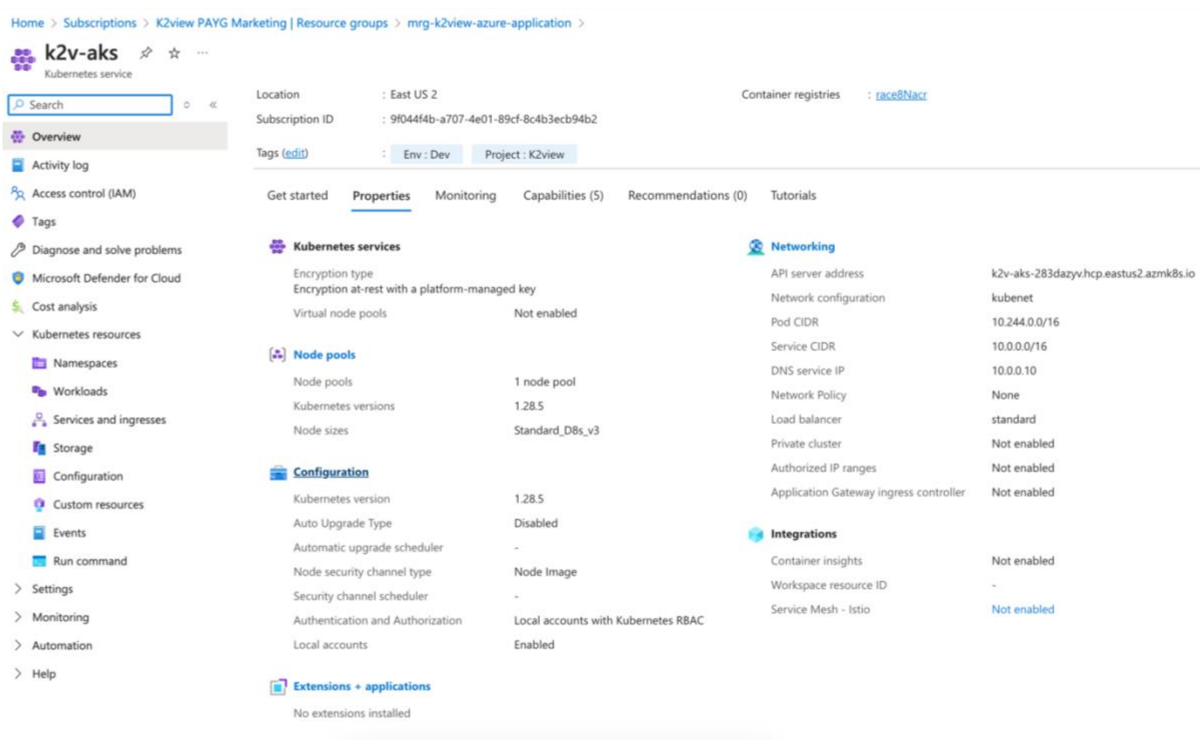
Expiration

NOTE: Certificates have expiration dates. Please note the date of expiration of the certificate provided to you and replace it before the expiration date as described next.

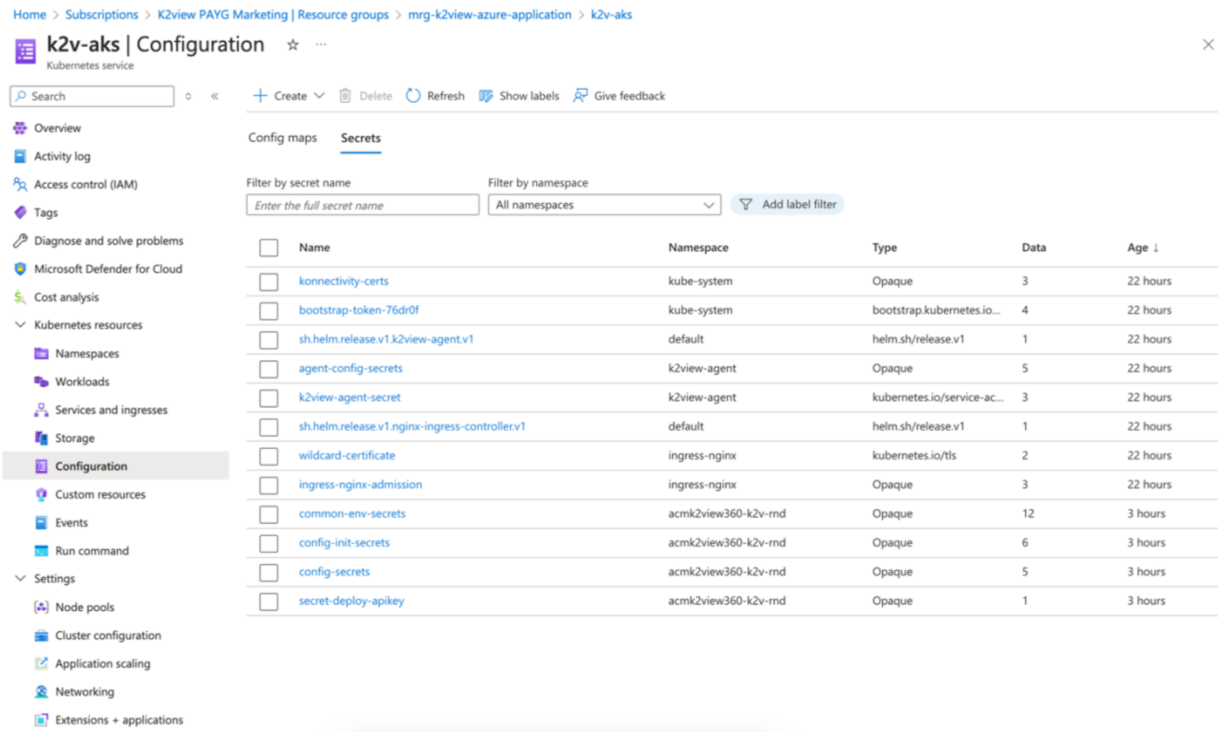
Replacing Certificates

If you encounter certificate issues or need to update your certificate, follow these instructions. You can also make a support request with K2view for assistance.

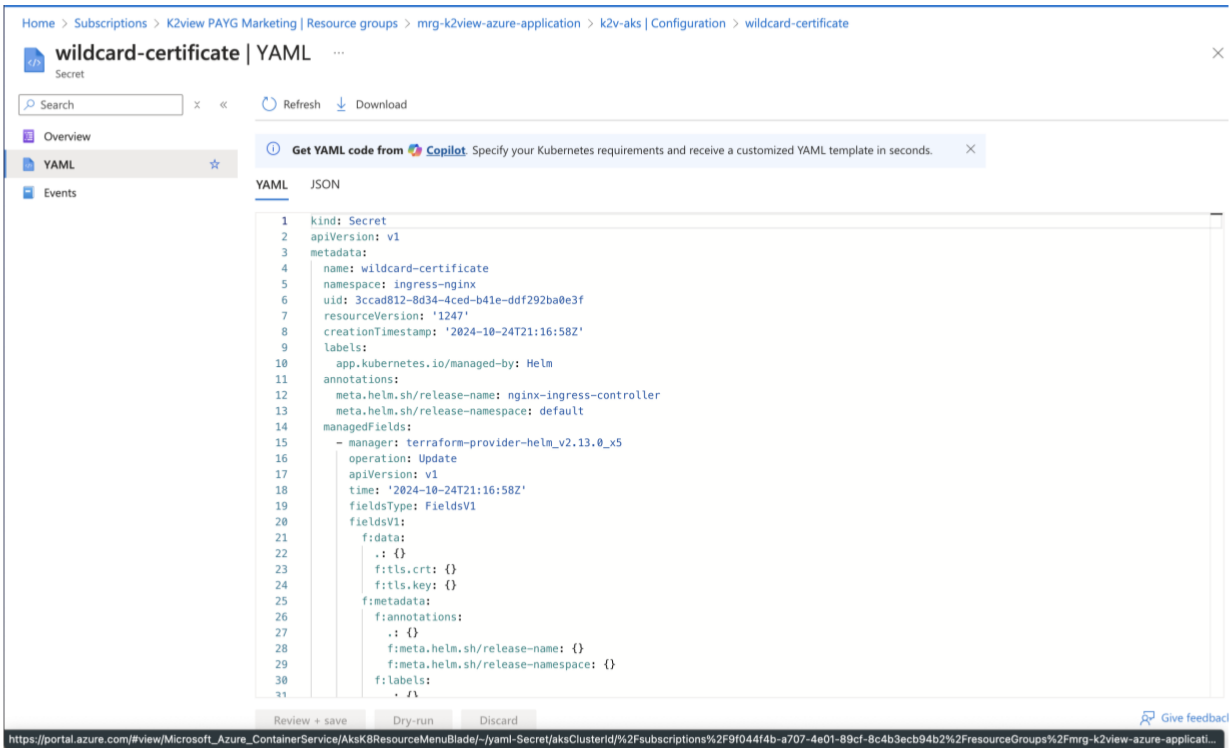
First, select the Managed Azure Application resource group created by the installation. Select the Kubernetes service item.



Then select the “Configuration” left menu item as shown here, select the Secrets tab, and then select the “wildcard-certificate”.



After opening the wildcard certificate and selecting the YAML section, scroll down and locate the Key and Certificate sections and paste replacement values (not shown here). These values are the base64 encoded values of your PEM certificate and private key.



Your PEM TLS certificate and its associated private key

Your network team must create a wildcard TLS certificate for the domain you selected as your Domain Ingress. The full certificate authority chain needs to be present. During the installation, you must paste the base64-encoded values of your TLS certificate and its associated private key.

- **TLS Private**: the base64-encoded private key associated with the TLS certificate
- **TLS Certificate**: a base64-encoded wildcard certificate. For example, for the domain ingress of acm.cloud.k2view.com you need to create a wildcard certificate for *.acm.cloud.k2view.com for hosts of this domain.

What you paste in should look like the following:

base64-encoded TLS Private Key:


```
LS0tLS1CRUdJTiBQUklWQVRFIEtFWS0tLS0tCk1JSUpRd0lCQURBTkJna3Foa2lHOXcwQkFRRUZBQVNDQ1Mwd2dna3BBZ
0VBQW9JQ0FRRE9yakUrN0NQVkU2QVgKV0lFRFVvdnVLc0lXOEJ4MTR
L…
QTlZV1pKKzlqRkhFUTdBZXB3S1RtMEVraz0KLS0tLS1FTkQgUFJJVkFURSBLRVktLS0tLQo=
```


base64-encoded TLS Certificate:


```
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSUdGRENDQlB5Z0F3SUJBZ0lTQS9YVFJKaUpXOXMzbldZZlN5a0RpZ
lh5TUEwR0NTcUdTSWIzRFFFQkN3VUEKTURNeEN6QUpCZ05WQkFZVEFsVlRNUll3RkFZRFZRUUtFdzFNWlhRbm
…
1ND0KLS0tLS1FTkQgQ0VSVElGSUNBVEUtLS0tLQ==
```


Note: The PEM certificate and key must be further encoded using base64 as input to the installation screen. To encode a PEM certificate (e.g., cert.pem) to base64 on both Linux and Windows, you can use the following commands.

Linux: You can use the base64 command to read the cert.pem file, encodes it in base64, and saves the output to cert_base64.txt.

```
base64 cert.pem > cert_base64.txt
```

Windows: On Windows, you can use the certutil command to encode the cert.pem file in base64 and saves the result to cert_base64.txt.

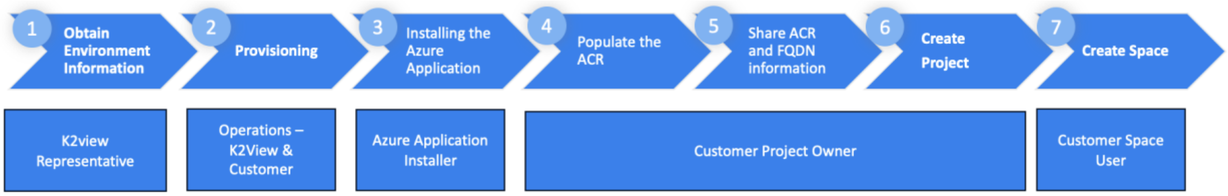
```
certutil -encode cert.pem cert_base64.txt
```

Prerequisite: You should prepare these base64 encoded files before the start of the installation.

10. Installation Worksheet

Please consult the Prerequisites section before performing the installation.

At various steps during the installation, you must have obtained information from K2view and, in Step 5, provided information to K2view.



The following summarizes the information you and K2view require. Ensure that Azure resources are registered by a subscription owner before proceeding to the installation of the K2view Azure Application.

[ ] K2view Admin user:

____

Provisioned on your behalf by K2view. Please consult with your K2view representative for this information.

[ ] Azure Region for the K2view Azure Application Installation: ______________________________

[ ] Azure Application Resource Group name: ______________________________

[ ] Azure Managed Application Resource Group name: ______________________________

[ ] Azure AKS Cluster name: ______________________________

[ ] Azure Environment Tag name: ______________________________

[ ] Azure Project Tag name: ______________________________

[ ] Azure Project Tag name: ___________________________

[ ] Domain Ingress FQDN: ___________________________

[ ] Base64-encoded Wildcard TLS PEM certificate: This information will be in a file

Full CA chain to be included

[ ] Base64-encoded TLS PEM Private Key: This information will be in a file

[ ] K2view Cloud Mailbox ID provided to you by K2view: ___________________________

[ ] K2view Nexus Repository Account supplied by K2view: ___________________________

[ ] List of Docker images to pull from Nexus:

Recommended by K2view. Please consult with your K2view representative for this information.

____

____

____

[ ] List of Docker images to pushed to your Azure Container Registry:

This list needs to be shared with K2view.

____

____

____

After the completion of Step 5 - the image population of your ACR is complete, and before the creation of your K2view Project and your first Space, you need to provide K2view with. Once confirmed you will be able to create your K2view Project:

[ ] FQDN of your Domain Ingress: ___________________________

Can be shared with K2view before the installation if you have selected the domain

[ ] ACR locations of your Docker images: ___________________________

The ACR created by the K2view installer generates a random name. You will need to provide this information after your installation. If you plan to use a pre-existing container registry, this information can be provided earlier.

[ ] AKS Cluster name: ___________________________

K2view will use the default value of k2v-aks

K2view Project Creation Step

You will need the following information to create your project.

[ ] Git Repository URL: ___________________________

[ ] Git token (do not share): ___________________________

[ ] Git Branch / Tag: ___________________________

11. Troubleshooting

DNS Issues

To start, please refer to the DNS zone section. For assistance, please make a support request with K2view.